Assignment 1.1

Name: Gunasekaran Pasupathy Date: May 10, 2026

For this assignment, you will refer to the textbook to solve the practice exercises. **Use Python to answer any coding problems (not R, even if indicated in your textbook).** Use Jupyter Notebook, Google Colab, or a similar software program to complete your assignment. Submit your answers as a **PDF or HTML** file. As a best practice, always label your axes and provide titles for any graphs generated on this assignment. Round all quantitative answers to 2 decimal places.

Problem # 1.1.

In the 2018 election for Senate in California, a CNN exit poll of 1882 voters stated that 52.5% voted for the Democratic candidate, Diane Feinstein. Of all 11.1 million voters, 54.2% voted for Feinstein.

(a) What was the (i) subject, (ii) sample, (iii) population?

`(a) Your answer goes here`

In [1]:
```
# Exit poll size = 1882
# poll result -> F got 52.5%, Actual result of 11.1 M votes , 54.2%

#### Answers #####
# Subject = California Voter
# sample  1882 Votes in California
# population is 11.1 M voters in California
```

Problem # 1.2.

The `Students` data file at http://stat4ds.rwth-aachen.de/data/Students.dat responses of a class of 60 social science graduate students at the University of Florida to a questionnaire that asked about *gender* (1 = female, 0 = male), *age*, *hsgpa* = high school GPA (on a four-point scale), cogpa = college GPA, *dhome* = distance (in miles) of the campus from your home town, *dres* = distance (in miles) of the classroom from your current residence, *tv* = average number of hours per week that you watch TV, *sport* = average number of hours per week that you participate in sports or have other physical exercise, *news* = number of times a week you read a newspaper, *aids* = number of people you know who have died from AIDS or who are HIV+, *veg* = whether you are a vegetarian (1 = yes, 0 = no), *affil* = political affiliation (1 = Democrat, 2 = Republican, 3 = independent), *ideol* = political ideology (1 = very liberal, 2 = liberal, 3 = slightly liberal, 4

= moderate, 5 = slightly conservative, 6 = conservative, 7 = very conservative), *relig* = how often you attend religious services (0 = never, 1 = occasionally, 2 = most weeks, 3 = every week), *abor* = opinion about whether abortion should be legal in the first three months of pregnancy (1 = yes, 0 = no), *affirm* = support affirmative action (1 = yes, 0 = no), and *life* = belief in life after death (1 = yes, 2 = no, 3 = undecided). You will use this data file for some exercises in this book.

(a) Practice accessing a data file for statistical analysis with your software by going to the book's website and copying and then displaying this data file.

`(a) Your answer goes here`

In [16]:
```
from os.path import sep

import pandas as pd
from pandas.core.interchange import column

# read the students data using pandas
students = pd.read_csv('https://stat4ds.rwth-aachen.de/data/Students.dat', s

print(students)

# students.head()
# students.describe()
# subject = social science graduate student at University of FL
# sample size = 60

## Used ChatGPT to see what other interesting functions that are available i
# students.tail()
# students.info()
# print(students.columns)
# print(students.dtypes)
##  Transposes the table -> good for readability
# students.describe().T
## finding the count of missing values
# print (students.isnull().sum())
## Finding unique values
# print(students['gender'].unique())
## retuns 3 random rows
# print(students.sample())
## Correlation matrix
# print(students.corr())
```

	subject	gender	age	hsgpa	cogpa	dhome	dres	tv	sport	news	aids
0	1	0	32	2.2	3.5	0	5.00	3.0	5	0	0
1	2	1	23	2.1	3.5	1200	0.30	15.0	7	5	6
2	3	1	27	3.3	3.0	1300	1.50	0.0	4	3	0
3	4	1	35	3.5	3.2	1500	8.00	5.0	5	6	3
4	5	0	23	3.1	3.5	1600	10.00	6.0	6	3	0
5	6	0	39	3.5	3.5	350	3.00	4.0	5	7	0
6	7	0	24	3.6	3.7	0	0.20	5.0	12	4	2
7	8	1	31	3.0	3.0	5000	1.50	5.0	3	3	1
8	9	0	34	3.0	3.0	5000	2.00	7.0	5	3	0
9	10	0	28	4.0	3.1	900	2.00	1.0	1	2	1
10	11	0	23	2.3	2.6	253	1.50	10.0	15	1	1
11	12	1	27	3.5	3.6	190	3.00	14.0	3	7	0
12	13	0	36	3.3	3.5	245	1.50	6.0	15	12	5
13	14	0	28	3.2	3.2	500	6.00	3.0	10	1	2
14	15	1	28	3.0	3.5	3500	1.00	4.0	3	1	0
15	16	1	25	3.8	3.3	210	10.00	7.0	6	1	0
16	17	1	41	4.0	3.0	1000	15.00	6.0	7	3	1
17	18	0	50	3.8	3.8	0	3.00	5.0	9	6	1
18	19	0	71	4.0	3.5	5000	3.00	6.0	12	2	2
19	20	1	28	3.0	3.8	120	1.00	25.0	0	0	2
20	21	1	26	3.7	3.7	8000	8.00	4.0	4	4	1
21	22	1	27	4.0	3.7	2	2.50	4.0	2	7	0
22	23	0	31	2.7	3.5	1700	5.00	7.0	7	2	0
23	24	1	23	3.7	3.7	2	2.00	7.0	4	2	0
24	25	0	23	3.2	3.8	450	4.00	0.0	7	7	3
25	26	1	44	3.0	3.0	0	2.00	2.0	3	2	3
26	27	0	26	3.7	3.0	1000	3.00	8.0	2	7	0

27	28	1	31	3.7	3.8	850	10.00	10.0	3	7	0
28	29	0	24	3.3	3.1	420	2.00	10.0	6	5	0
29	30	1	26	3.3	3.3	1200	0.75	10.0	0	3	0
30	31	0	26	3.3	3.5	1000	1.50	0.0	3	3	3
31	32	1	32	3.5	3.9	150	12.00	10.0	2	0	0
32	33	0	26	3.4	3.4	2000	1.50	2.0	7	14	0
33	34	1	22	3.2	2.8	316	2.00	10.0	3	5	2
34	35	1	24	3.5	3.9	900	1.75	8.0	0	0	1
35	36	0	24	3.6	3.3	250	2.00	4.0	6	3	1
36	37	0	23	3.8	3.7	180	0.50	10.0	5	7	0
37	38	0	33	3.4	3.4	6000	1.50	8.0	5	6	2
38	39	0	23	2.8	3.2	950	2.00	37.0	10	5	0
39	40	0	31	3.8	3.5	1100	0.75	0.5	3	5	2
40	41	0	26	3.4	3.4	1300	1.20	0.0	8	2	0
41	42	0	28	2.0	3.0	360	0.25	10.0	8	3	0
42	43	1	24	3.8	3.9	1800	2.00	2.0	5	4	1
43	44	0	23	3.0	3.6	900	15.00	12.0	0	5	0
44	45	1	25	3.0	4.0	5000	5.00	1.5	0	4	0
45	46	1	24	3.0	3.5	300	1.00	10.0	5	5	0
46	47	1	27	3.0	3.8	2000	20.00	28.0	7	14	2
47	48	0	24	3.3	3.8	630	1.30	2.0	3	5	0
48	49	1	26	3.8	4.0	1200	1.00	0.0	4	3	1
49	50	1	27	3.0	4.0	580	2.00	5.0	15	1	2
50	51	0	32	3.0	3.0	2000	5.00	5.0	5	2	1
51	52	1	41	4.0	4.0	0	8.00	8.0	4	2	2
52	53	1	29	3.0	3.9	300	3.70	2.0	5	1	1 1
53	54	1	50	3.5	3.8	6	6.00	7.0	3	7	0
54	55	1	22	3.4	3.7	80	7.00	10.0	1	2	2

```
55       56       1   23    3.6    3.2    375   1.50   5.0      10      5
0
56       57       0   26    3.5    3.6   2000   0.30  16.0       8      3
0
57       58       0   30    3.0    3.0      1   1.10   1.0       4      3
0
58       59       1   23    3.0    3.0    112   0.50  15.0       3      3
0
59       60       1   22    3.4    3.0    650   4.00   8.0      16      7
1
```

	veg	affil	ideol	relig	abor	affirm	life
0	0	2	6	2	0	0	1
1	1	1	2	1	1	1	3
2	1	1	2	2	1	1	3
3	0	3	4	1	1	1	2
4	0	3	1	0	1	0	2
5	1	1	2	1	1	1	3
6	0	3	2	1	1	1	1
7	0	3	2	1	1	1	1
8	0	3	1	1	1	1	3
9	1	3	3	0	0	1	1
10	0	2	5	1	0	1	1
11	0	1	2	1	1	1	3
12	0	1	1	1	1	1	1
13	0	3	4	1	1	0	1
14	0	1	1	0	1	1	1
15	1	3	2	3	1	1	1
16	0	3	3	3	0	0	1
17	0	1	2	0	1	0	2
18	0	3	2	0	1	0	2
19	1	1	1	1	1	1	1
20	0	3	4	1	1	1	1
21	0	3	2	1	1	1	1
22	0	2	7	3	0	0	1
23	0	3	4	0	1	1	1
24	0	3	1	0	1	1	1
25	1	3	3	2	1	1	1
26	0	1	2	1	1	1	3
27	0	2	5	2	1	0	1
28	0	1	4	1	1	1	3
29	0	2	2	1	1	1	3
30	1	1	2	1	1	1	2
31	0	1	2	1	0	0	1
32	0	1	2	0	1	1	2
33	0	3	2	1	1	1	3
34	0	1	1	1	1	1	3
35	0	2	5	3	0	1	1
36	0	3	2	0	1	0	3
37	0	3	2	0	1	1	2
38	0	2	5	2	1	0	1
39	0	2	6	2	1	0	3
40	0	3	2	1	0	1	2
41	0	1	3	0	1	1	3
42	0	2	6	3	0	1	1
43	0	2	5	0	1	0	2

```
44    0       3       4       1       1       1       2
45    0       1       2       0       1       1       2
46    1       2       3       1       1       1       1
47    0       2       7       3       0       0       1
48    0       1       2       0       1       1       2
49    0       1       1       1       1       1       2
50    0       2       5       3       0       1       1
51    0       2       4       1       0       0       1
52    0       1       2       1       1       1       1
53    0       1       2       1       1       1       3
54    0       3       2       0       1       1       3
55    0       2       6       3       0       0       1
56    0       1       4       1       1       1       3
57    0       3       3       3       1       0       1
58    0       3       4       2       1       1       1
59    0       3       4       1       1       1       1
```

(b) Using responses on *abor*, state a question that could be addressed with (i) descriptive statistics, (ii) inferential statistics.

Descriptive : what age group students support abortion the most? What percentage of females support abortion legality?

Inferential: Is there an association between hours of TV watched per week and opposition to abortion?

Problem # 1.3.

Identify each of the following variables as categorical or quantitative: (a) Number of smartphones that you own; (b) County of residence; (c) Choice of diet (vegetarian, nonvegetarian); (d) Distance, in kilometers, commute to work

- Number of Smartphones that you own - **quantitaive/discrete**
- Country of residence - **Catagorical/qualitative**
- Choice of diet - **categorical**
- distance - **quantitaive (continuous)**

Problem # 1.4.

Give an example of a variable that is (a) categorical; (b) quantitative; (c) discrete; (d) continuous

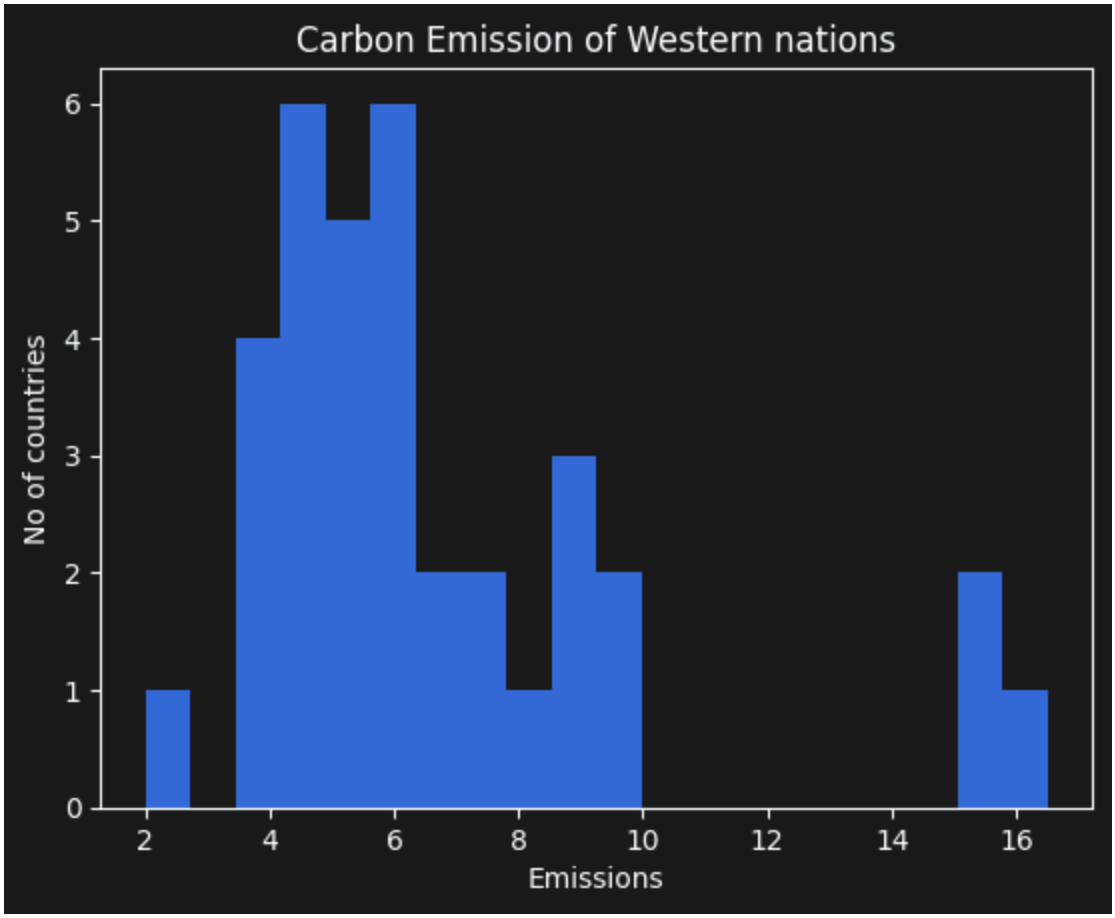
- Categorical - **Political Idealogy**
- quntitative - **age (and continuous assuming age can be a fraction)**
- discrete - **news**
- Continuous - **hsgpa, cogpa**

Problem # 1.10.

Analyze the `Carbon_West` (http://stat4ds.rwth-aachen.de/data/Carbon_West.dat) data file at the book's website by **(a)** constructing a frequency distribution and a histogram, **(b)** finding the mean, median, and standard deviation. Interpret each.

(a) constructing a frequency distribution and a histogram

In [56]:
```
# import
import pandas as pd
import matplotlib.pyplot as plt

# Load data
cw = pd.read_csv('https://stat4ds.rwth-aachen.de/data/Carbon_West.dat', sep=
# print(cw.columns) - so only Nation and CO2 are the column header
# cw.describe()
## plotting histogram using matplotlib
plt.title('Carbon Emission of Western nations')
plt.xlabel('Emissions')
plt.ylabel('No of countries')

### This felt like disconnected to plt.. so asked chatGpt for an alternate u
# cw['CO2'].hist( bins=50) - This is what I came up with.
counts, bins, patches = plt.hist(cw['CO2'], bins=20)  ## This is better
plt.show()

### I did not know if there were in built functions that are available in pa
# frequency distribution
for i in range(len(counts)):
    print("Range:", bins[i], "to", round(bins[i + 1], 2))
    print("Frequency:", int(counts[i]))

## Interpretation
# Most countries' CO2 emission levels fall b/w 2 and 10. A few countries are
```


Carbon Emission of Western nations
No of countries
Emissions
0
1
2
3
4
5
6
2
4
6
8
10
12
14
16

```
Range: 2.0 to 2.72
Frequency: 1
Range: 2.725 to 3.45
Frequency: 0
Range: 3.45 to 4.18
Frequency: 4
Range: 4.175 to 4.9
Frequency: 6
Range: 4.9 to 5.62
Frequency: 5
Range: 5.625 to 6.35
Frequency: 6
Range: 6.35 to 7.08
Frequency: 2
Range: 7.075 to 7.8
Frequency: 2
Range: 7.8 to 8.52
Frequency: 1
Range: 8.524999999999999 to 9.25
Frequency: 3
Range: 9.25 to 9.98
Frequency: 2
Range: 9.975 to 10.7
Frequency: 0
Range: 10.7 to 11.42
Frequency: 0
Range: 11.424999999999999 to 12.15
Frequency: 0
Range: 12.15 to 12.88
Frequency: 0
Range: 12.875 to 13.6
Frequency: 0
Range: 13.6 to 14.32
Frequency: 0
Range: 14.325 to 15.05
Frequency: 0
Range: 15.049999999999999 to 15.78
Frequency: 2
Range: 15.775 to 16.5
Frequency: 1
```

(b) Your answer goes here

In [55]:

```python
import numpy as np

# find mean, median and standard deviation
## prints everything
# cw.describe()


## since we want mean, median and standard deviation alone,
print("########### Using Pandas ###########")
print('mean', round(cw['CO2'].mean(), 2))
print('median', round(cw['CO2'].median(), 2))
print('std. dev.', round(cw['CO2'].std(), 2))
```

```
## Looked up internet to see if there are alternatives and found numpy has a
# and noticed that standard deviation had a slighly different value. Need to
print("########### Using Numpy ############")
print('mean', round(np.mean(cw['CO2']), 2))
print('median', round(np.median(cw['CO2']), 2))
print('std. dev.', round(np.std(cw['CO2']), 2))

### Looks like ties back to whether the formulae uses n vs n-1 in the denomi

# interpretation
# The aveerage emission across the western nations is 6.71
```

```
########### Using Pandas ###########
mean 6.72
median 5.9
std. dev. 3.36
########### Using Numpy ###########
mean 6.72
median 5.9
std. dev. 3.31
```

Problem # 1.11.

According to Statistics Canada, for the Canadian population having income in 2019, annual income had a median of $ 35,000 and mean of $ 46,700. What would you predict about the shape of the distribution? Why?

Annual Income

median = 35000 mean = 46700

Since the average (mean) is higher than mid point (median), I think it is a skewed shape (right skewed)

Or there is a big outlier contributing to a higher mean

Problem # 1.13.

A report indicates that public school teacher's annual salaries in New York city have an approximate mean of $ 69,000 and standard deviation of $ 6,000. If the distribution has approximately a bell shape, report intervals that contain about (a) 68%, (b) 95%, (c) all or nearly all salaries. Would a salary of $ 100,000 be unusual? Why?

mean = 69000 std dev = 6000 shape = bell

According to *Foundations of Statistics for Data Scientists* (2022), 68% would fall b/w (mean - std.dev.) and (mean + std.dev) which is 69000 + or - 6000

95% would fall b/w (mean - 2* sd) and (mean + 2 *sd) which is 69000 + or - 12000

Near 100% would fall b/w (mean - 3 * sd) and (mean +3*sd) , which is 69000 - or + 18000

so, it is unusal to have a 100k salary

Problem # 1.17.

From the `Murder` data file (http://stat4ds.rwth-aachen.de/data/Murder.dat) at the book's website, use the variable murder, which is the murder rate (per 100,000 population) for each state in the U.S. in 2017 according to the FBI Uniform Crime Reports. At first, do not use the observation for D.C. (DC). Using software:
(a) Find the mean and standard deviation and interpret their values.
(b) Find the five-number summary, and construct the corresponding box plot. Interpret.
(c) Now include the observation for D.C. What is affected more by this outlier: The mean or the median? The range or the inter-quartile range?

Answer:

`(a) Your answer goes here`

In [83]:
```
# (a) Find the mean and standard deviation and interpret their values. (DC c


# Murder rate per state - variable
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import gridspec

# read file

murder_rate_df = pd.read_csv('https://stat4ds.rwth-aachen.de/data/Murder.dat
# print(mr)

# # store DC data so it can be added later
# dc = mr[mr['state']=='DC']
# # print(dc.head())
#
# # exclude DC data and store it in the same variable
# mr = mr[mr['state']!='DC']

# May be create a new data frame excluding DC

murder_rate_df_excluding_dc = murder_rate_df[murder_rate_df['state']!='DC']

# fig.supxlabel('State')
# fig.supylabel('Murder Rate')
### DC excluded metrics
# 5 number summary
murder_rate_df_excluding_dc.describe().round(2)
# mean = 4.87
```

```
# std.dev = 2.59

#interpretation
print( 'The average murder rate in the US states (excluding the D.C.) is arc
```

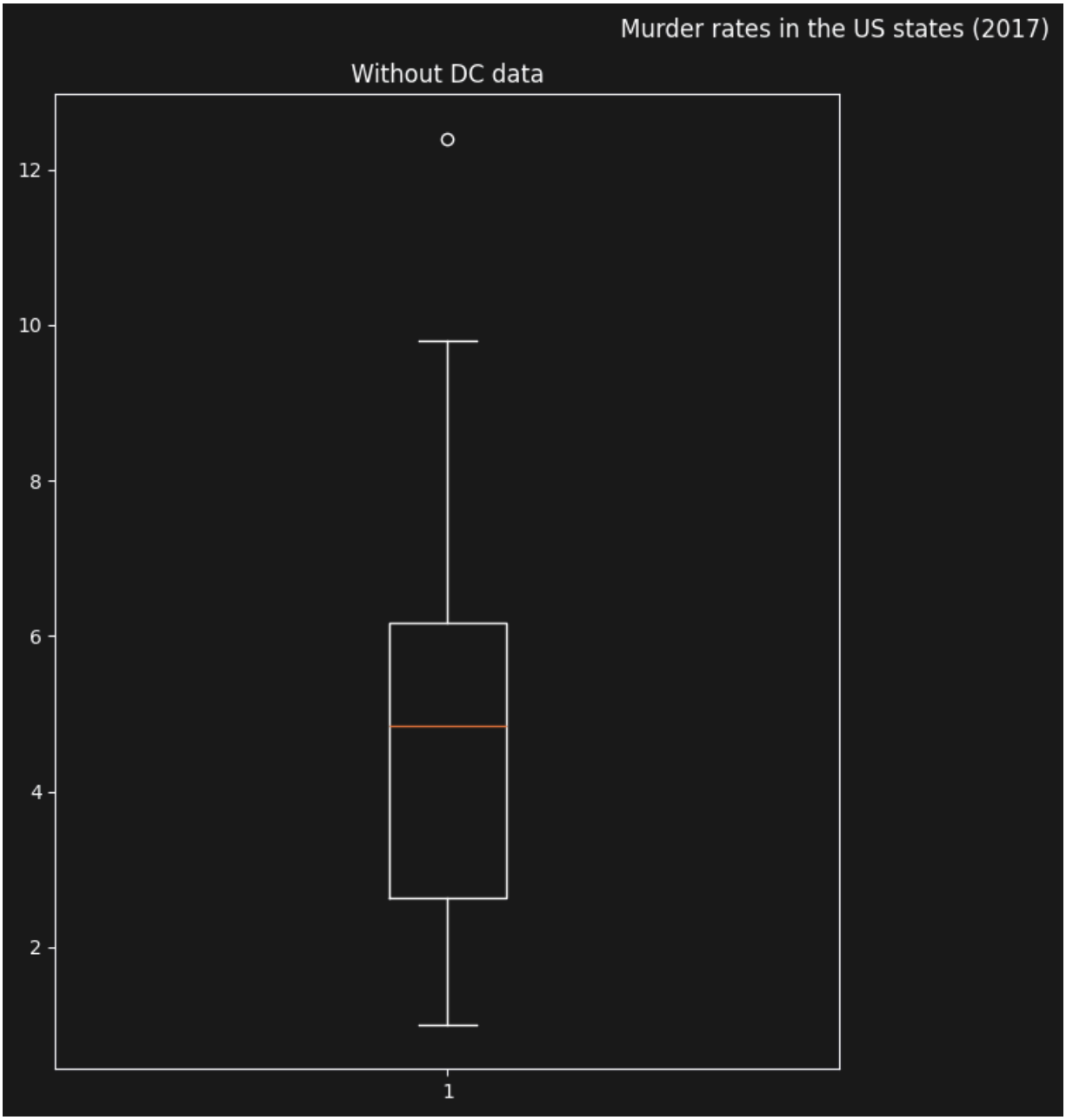
The average murder rate in the US states (excluding the D.C.) is around 4.87 per 100 K population with a moderate deviation of 2.59 among states

(b) Your answer goes here

In [84]:
```
# (b) Find the five-number summary, and construct the corresponding box plot
# include DC

#print 5 number summary and compare

#5 number summary
print (murder_rate_df_excluding_dc.describe().round(2))

gs = gridspec.GridSpec(1, 2)
fig = plt.figure(figsize=(12,8))
fig.suptitle('Murder rates in the US states (2017)')



# box plot
ax = fig.add_subplot(gs[0])
ax.set_title('Without DC data')
fig.tight_layout()
# murder_rate_df_excluding_dc.boxplot()
ax.boxplot(murder_rate_df_excluding_dc['murder'])
```

```
       murder
count   50.00
mean     4.87
std      2.59
min      1.00
25%      2.62
50%      4.85
75%      6.18
max     12.40
```

Out[84]:
```
{'whiskers': [<matplotlib.lines.Line2D at 0x115339550>,
  <matplotlib.lines.Line2D at 0x1153396a0>],
 'caps': [<matplotlib.lines.Line2D at 0x1153397f0>,
  <matplotlib.lines.Line2D at 0x115339940>],
 'boxes': [<matplotlib.lines.Line2D at 0x115339400>],
 'medians': [<matplotlib.lines.Line2D at 0x115339a90>],
 'fliers': [<matplotlib.lines.Line2D at 0x115339be0>],
 'means': []}
```



(c) Your answer goes here

In [99]:
```
# (c) Now include the observation for D.C. What is affected more by this out

# print(murder_rate_df_excluding_dc['murder'].median() )
# print(murder_rate_df_excluding_dc['murder'].mean() )
# print(murder_rate_df['murder'].median() )
# print(murder_rate_df['murder'].mean() )

range_with_dc = murder_rate_df['murder'].max() - murder_rate_df['murder'].mi
range_without_dc = murder_rate_df_excluding_dc['murder'].max() - murder_rate


# Looks like pandas only has quantile not quartile.. so using it

quartile_range_excluding_dc = (murder_rate_df_excluding_dc['murder'].quantil
                  - murder_rate_df_excluding_dc['murder'].quantile(0.25))
```

```
# print(quartile_range_excluding_dc.round(2))
#
quartile_range_with_dc = (murder_rate_df['murder'].quantile(0.75)
                  - murder_rate_df['murder'].quantile(0.25))
# print(quartile_range_with_dc.round(2))


## Asked Claude to print the above in more readable format.. learned DataFra

comparison_df = pd.DataFrame({
    'Dataset': ['Without DC', 'With DC'],

    'Mean': [
        round(murder_rate_df_excluding_dc['murder'].mean(), 2),
        round(murder_rate_df['murder'].mean(), 2)
    ],
    'Median': [
        round(murder_rate_df_excluding_dc['murder'].median(), 2),
        round(murder_rate_df['murder'].median(), 2)
    ],
    'IQR': [
        round(quartile_range_excluding_dc, 2),
        round(quartile_range_with_dc, 2)
    ],
    'Range': [
        round(range_without_dc, 2),
        round(range_with_dc, 2)
    ]
})

print(comparison_df)

print('Interpretation \n')
print ('Mean vs Median: The median changed a little compared to the mean, sc
print ('Range vs IQR: IQR increased a little while range difference was dras

## side by side box plot
# gs = gridspec.GridSpec(1, 2)
# fig = plt.figure(figsize=(12,8))
# fig.suptitle('Murder rates in the US states (2017)')


# # box plot
# ax = fig.add_subplot(gs[0])
# ax.set_title('Without DC data')
# fig.tight_layout()
# # murder_rate_df_excluding_dc.boxplot()
# ax.boxplot(murder_rate_df_excluding_dc['murder'])
#
# ax= fig.add_subplot(gs[1])
# ax.set_title('With DC data')
# fig.tight_layout()
# # murder_rate_df_excluding_dc.boxplot()
# ax.boxplot(murder_rate_df['murder'])
```

```
      Dataset  Mean  Median   IQR  Range
0  Without DC  4.87    4.85  3.55   11.4
1     With DC  5.25    5.00  3.80   23.2
Interpretation

Mean vs Median: The median changed a little compared to the mean, so the DC
murder rate is as an outlier (high)

Range vs IQR: IQR increased a little while range difference was drastic, sug
gesting that the DC data is an extreme outlier
```

Problem # 1.18.

The `Income` data file (http://stat4ds.rwth-aachen.de/data/Income.dat) at the book's website reports annual income values in the U.S., in thousands of dollars.

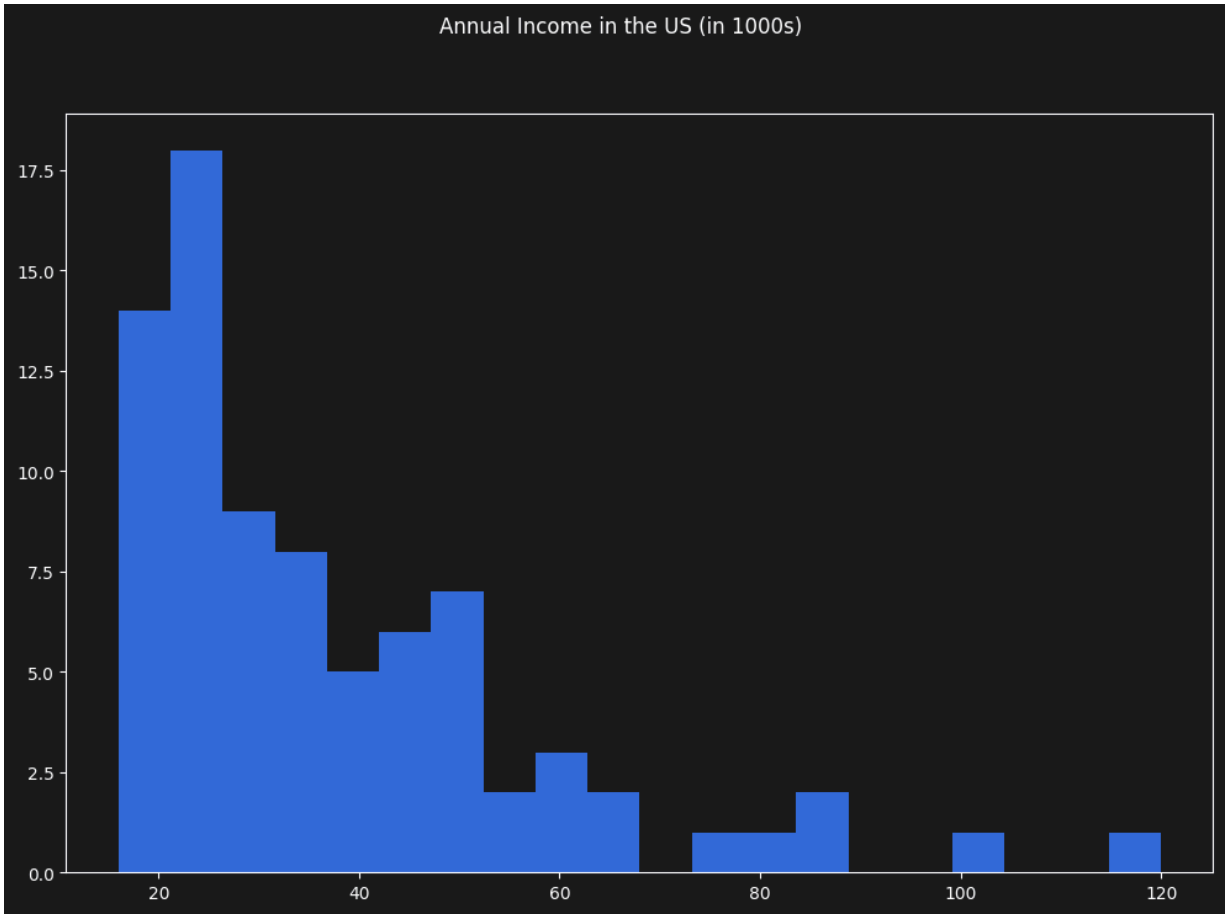
(a) Using software, construct a histogram. Describe its shape.
(b) Find descriptive statistics to summarize the data. Interpret them.
(c) The kernel density estimation method finds a smooth-curve approximation for a histogram. At each value, it takes into account how many observations are nearby and their distance, with more weight given those closer. Increasing the bandwidth increases the influence of observations further away. Plot a smooth-curve approximation for the histogram of income values. Summarize the impact of increasing and of decreasing the bandwidth substantially from the default value.
(d) Construct and interpret side-by-side box plots of income by race (B = Black, H = Hispanic, W = White). Compare the incomes using numerical descriptive statistics

(a) Using software, construct a histogram. Describe its shape.

The histogram suggests that it is a right skewed distribution

In [101…
```
import pandas as py
import matplotlib.pyplot as plt
from matplotlib import gridspec

df = py.read_csv('https://stat4ds.rwth-aachen.de/data/Income.dat', sep='\\s+

gs = gridspec.GridSpec(1, 1)
fig = plt.figure(figsize=(12,8))
fig.suptitle('Annual Income in the US (in 1000s)')

ax = fig.add_subplot(gs[0])
ax.hist(df['income'], bins=20)

print('The histogram shape is right skewed')
```

```
The histogram shape is right skewed
```



(b) Find descriptive statistics to summarize the data. Interpret them.

Based on the data set,

Income

The average household income in the US is approximately 37,520. The middle 50% of the incomes fall between 22,000 and 46,500, indicating significant variation in income levels. The maximum observed income is 120,000, while the minimum is 16,000, suggesting noticeable income disparity in the country. The distribution is right-skewed because the mean is higher than the median (the histogram confirms it)

Education

The mean value of 12.69 suggests that the average person has completed high school. The education levels range from 7 to 20 years.

Race

The data set was collected from more whites than Black and Hispanics combined.

In [129…

```
# frequency distribution
## Quantitative
# print(df.describe().round(2).T)

print('Income summary\n', df['income'].describe().round(2))
print('Education Summary\n', df['education'].describe().round(2))
```

```
## Catagorical
df['race'].value_counts().sort_values()
# print (df.value_counts('income').sort_values())
# print (df.value_counts('race').sort_values())
# print (df.value_counts('education').sort_values())
```

```
Income summary
 count     80.00
mean      37.52
std       20.67
min       16.00
25%       22.00
50%       30.00
75%       46.50
max      120.00
Name: income, dtype: float64
Education Summary
 count    80.00
mean     12.69
std       2.87
min       7.00
25%      10.00
50%      12.00
75%      15.00
max      20.00
Name: education, dtype: float64
```

Out[129…

```
race
H    14
B    16
W    50
Name: count, dtype: int64
```

(c) The kernel density estimation method finds a smooth-curve approximation for a histogram. At each value, it takes into account how many observations are nearby and their distance, with more weight given those closer. Increasing the bandwidth increases the influence of observations further away. Plot a smooth-curve approximation for the histogram of income values. Summarize the impact of increasing and of decreasing the bandwidth substantially from the default value.

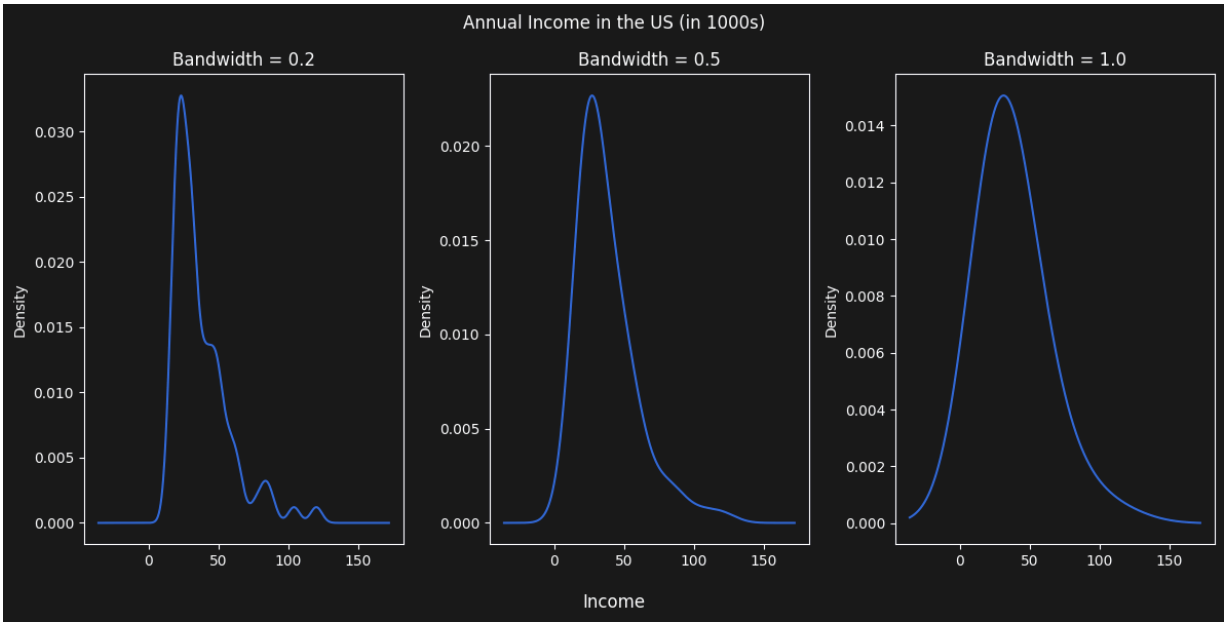
Answer Disclaimer - I am not sure if I missed "kernal density estimation" in the books and/or lecture. So, used internet to understand what it meant. - Looks like it is just a different plot type like bar graph.

Observation The higher the bandwidth, the smoother the curve becomes. However, large bandwidth values may hide important features and patterns in the distribution.

In [148…

```
gs = gridspec.GridSpec(1, 3)
fig = plt.figure(figsize=(12,6))
fig.suptitle('Annual Income in the US (in 1000s)')
ax = fig.add_subplot(gs[0])
ax.set_title('Bandwidth = 0.2')
df['income'].plot(kind='kde', bw_method=0.2)
```

```
ax=fig.add_subplot(gs[1])
ax.set_title('Bandwidth = 0.5')
df['income'].plot(kind='kde', bw_method=0.5)
ax=fig.add_subplot(gs[2])
ax.set_title('Bandwidth = 1.0')
df['income'].plot(kind='kde', bw_method=1.0)

fig.supxlabel('Income')
fig.tight_layout()
```



(d) Construct and interpret side-by-side box plots of income by race (B = Black, H = Hispanic, W = White). Compare the incomes using numerical descriptive statistics

The White population has the highest average and median income among the three groups, with a mean income of 42,480 and a median income of 37,000. The White group also has the greatest disparity in income, as shown by the largest standard deviation (22.87) and the highest maximum income (120,000).

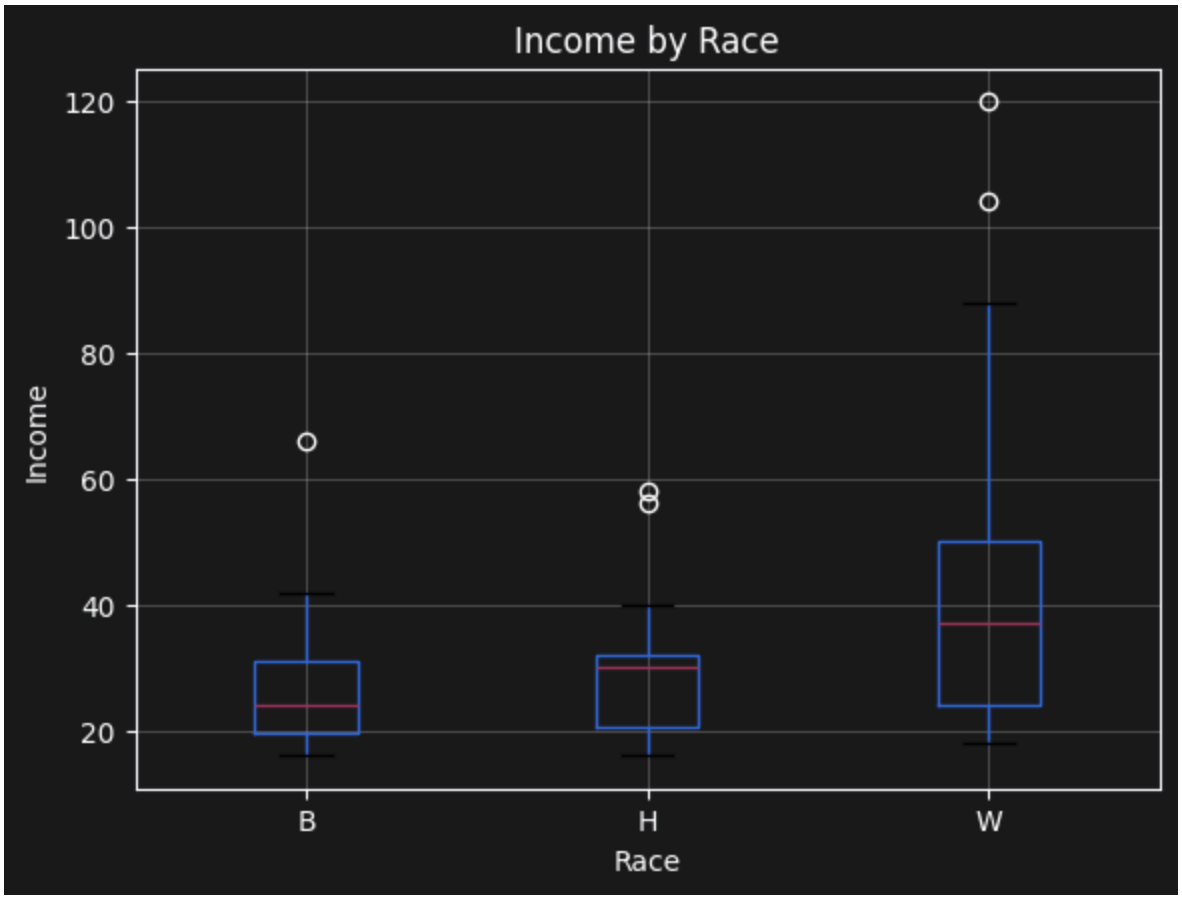
The Hispanic population has a higher median income (30,000) than the Black population (24,000), although both groups have relatively similar standard deviation. The Black population has the lowest median and mean income among the three groups.

The larger standard deviation and maximum value for the White population show greater income dispersion and the presence of extreme outliers.

In [157…

```
# gs = gridspec.GridSpec(1, 3)
# fig = plt.figure(figsize=(12,6))
# fig.suptitle('Race wise Annual Income in the US (in 1000s)')
#
# ax = fig.add_subplot(gs[0])
# ax.set_title('Income of Black population')
# #### DISCLAIMER : I had to use internet to find the right syntax for the b
# df[df['race']=='B']['income'].plot(kind='box')
#
```

```
# ax = fig.add_subplot(gs[1])
# ax.set_title('Income of White population')
# df['income'].plot(kind='box')
#
# ax = fig.add_subplot(gs[2])
# ax.set_title('Income of Haspanic population')
# df['income'].plot(kind='box')
# fig.tight_layout()
#
# fig.tight_layout()

# DISCLAIMER: I cross checked my solution with calude and found an easier ar

df.boxplot(column='income', by='race')
plt.title('Income by Race')
plt.suptitle('')
plt.xlabel('Race')
plt.ylabel('Income')
plt.show()

## numeric descriptive statistics

df.groupby('race')['income'].describe().round(2)
```



Out[157…

race	count	mean	std	min	25%	50%	75%	max
B	16.0	27.75	13.28	16.0	19.5	24.0	31.0	66.0
H	14.0	31.00	12.81	16.0	20.5	30.0	32.0	58.0
W	50.0	42.48	22.87	18.0	24.0	37.0	50.0	120.0

Problem # 1.19.

The `Houses` data file (http://stat4ds.rwth-aachen.de/data/Houses.dat) at the book's website lists the selling price (thousands of dollars), size (square feet), tax bill (dollars), number of bathrooms, number of bedrooms, and whether the house is new (1 = yes, 0 = no) for 100 home sales in Gainesville, Florida. Let's analyze the selling prices.

(a) Construct a frequency distribution and a histogram. Describe the shape.
(b) Find the percentage of observations that fall within one standard deviation of the mean. Why is this not close to 68%?
(c) Construct a box plot, and interpret.
(d) Use descriptive statistics to compare selling prices according to whether the house is new.

Answer:

(a) Construct a frequency distribution and a histogram. Describe the shape.

price - right skewed

Size - right skewed

Tax - right skewed

Bedrooms and Baths - Looks like bell shaped with extended tail (no sure if there is a shape for this)

new - it is discreate binary, so shape does not make sense

In [186…

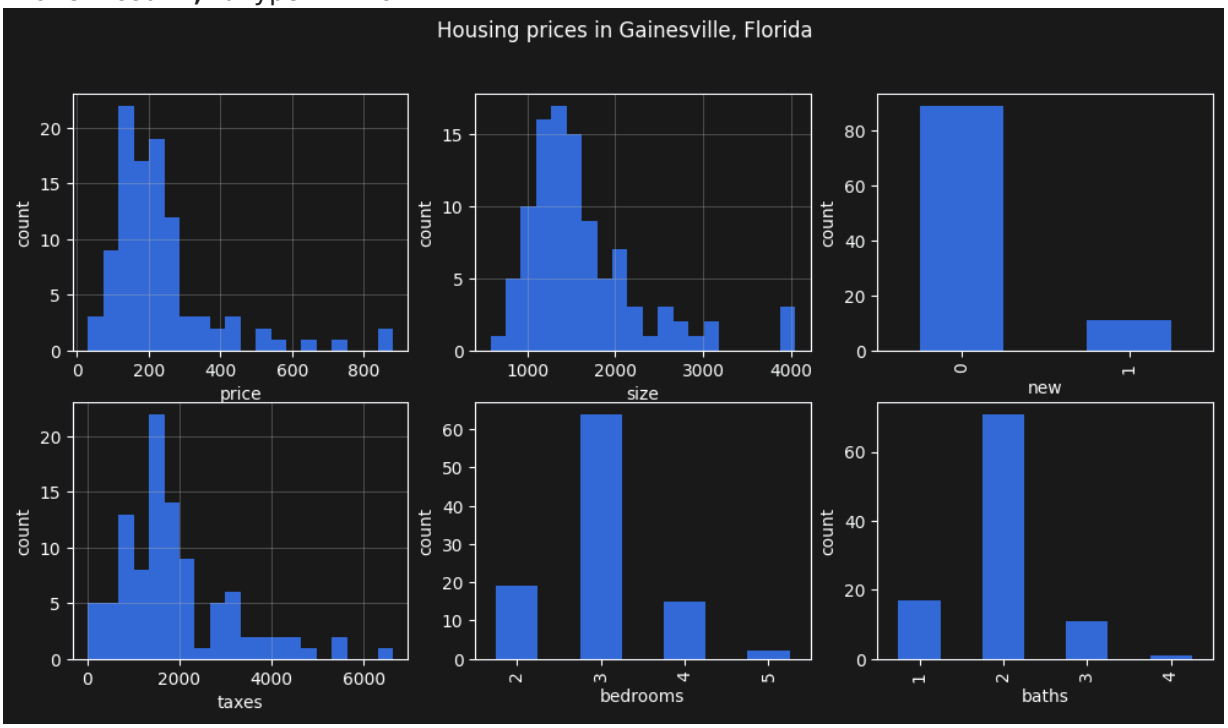
```
df_house = pd.read_csv('https://stat4ds.rwth-aachen.de/data/Houses.dat', sep

column_names = df_house.columns
column_names = column_names.drop('case')
n_cols = len(df_house.columns)
v_cols = 3 # 3 rows
v_rows = (int(n_cols / v_cols))
```

```
discrete_columns = ['new','bedrooms','baths']

gs = gridspec.GridSpec(v_rows,v_cols)
fig = plt.figure(figsize=(12,6))
fig.suptitle('Housing prices in Gainesville, Florida')
k = 0
for i in range(v_rows):
    # we need this if no of columns is not a multiple of 3 (but not required
    if k >= n_cols:
        break
    for j in range(v_cols):
        current_col = column_names[k]
        ax=fig.add_subplot(gs[i,j])
        ax.set_xlabel(current_col)
        ax.set_ylabel('count')
        # df_house[current_col].hist()
        if column_names[k] in discrete_columns:
            temp =df_house[current_col].value_counts().sort_index()
            print ('Frequency distribution of the Discrete column: ', currer
            print (temp)
            temp.plot(kind='bar', ax=ax)
        else:
            # df_house[current_col].hist(ax=ax, bins=20)
            freq = pd.cut(df_house[current_col], bins=20).value_counts().sor
            print ('Frequency distribution of the continuous column: ', curr
            print(freq)
            df_house[current_col].hist(ax=ax, bins=20)
        k += 1



### First tried the below and realized this can be iterated and came up with
## Frequency distribution


## Discreate variables
# print (df_house['new'].value_counts().sort_index())
# print (df_house['bedrooms'].value_counts().sort_index())
# print (df_house['baths'].value_counts().sort_index())



# price - continuous
# size - continuous
# new - discrete, categorical
# taxes - continuous
# bedrooms - discrete
# baths - discrete


# gs = gridspec.GridSpec(2,3)
# fig = plt.figure(figsize=(12,6))
# fig.suptitle('Housing prices in Gainesville, Florida')
#
#
# ax=fig.add_subplot(gs[0,0])
```

```
# ax.set_xlabel('Price Range')
# ax.set_ylabel('count')
# df_house['price'].hist()
#
# ax=fig.add_subplot(gs[0,1])
# ax.set_xlabel('size')
# ax.set_ylabel('count')
# df_house['size'].hist()
#
# ax=fig.add_subplot(gs[0,2])
# ax.set_xlabel('New/Old')
# ax.set_ylabel('count')
# df_house['new'].hist()
#
# ax=fig.add_subplot(gs[1,0])
# ax.set_xlabel('Tax Range')
# ax.set_ylabel('count')
# df_house['taxes'].hist()
#
# ax=fig.add_subplot(gs[1,1])
# ax.set_xlabel('No. of bedrooms')
# ax.set_ylabel('count')
# df_house['bedrooms'].hist()
#
# ax=fig.add_subplot(gs[1,2])
# ax.set_xlabel('No. of baths')
# ax.set_ylabel('count')
# df_house['baths'].hist()
#
# fig.tight_layout()
```

```
Frequency distribution of the continuous column:  price
price
(30.651, 73.95]     3
(73.95, 116.4]      9
(116.4, 158.85]    22
(158.85, 201.3]    17
(201.3, 243.75]    19
(243.75, 286.2]    12
(286.2, 328.65]     3
(328.65, 371.1]     3
(371.1, 413.55]     2
(413.55, 456.0]     3
(456.0, 498.45]     0
(498.45, 540.9]     2
(540.9, 583.35]     1
(583.35, 625.8]     0
(625.8, 668.25]     1
(668.25, 710.7]     0
(710.7, 753.15]     1
(753.15, 795.6]     0
(795.6, 838.05]     0
(838.05, 880.5]     2
Name: count, dtype: int64
Frequency distribution of the continuous column:  size
size
(576.53, 753.5]      1
(753.5, 927.0]       5
(927.0, 1100.5]     10
(1100.5, 1274.0]    16
(1274.0, 1447.5]    17
(1447.5, 1621.0]    15
(1621.0, 1794.5]     9
(1794.5, 1968.0]     5
(1968.0, 2141.5]     7
(2141.5, 2315.0]     3
(2315.0, 2488.5]     1
(2488.5, 2662.0]     3
(2662.0, 2835.5]     2
(2835.5, 3009.0]     1
(3009.0, 3182.5]     2
(3182.5, 3356.0]     0
(3356.0, 3529.5]     0
(3529.5, 3703.0]     0
(3703.0, 3876.5]     0
(3876.5, 4050.0]     3
Name: count, dtype: int64
Frequency distribution of the Discrete column:  new
new
0    89
1    11
Name: count, dtype: int64
Frequency distribution of the continuous column:  taxes
taxes
(13.393, 350.35]      5
(350.35, 680.7]       5
(680.7, 1011.05]     13
```

```
(1011.05, 1341.4]     8
(1341.4, 1671.75]    22
(1671.75, 2002.1]    14
(2002.1, 2332.45]     9
(2332.45, 2662.8]     1
(2662.8, 2993.15]     5
(2993.15, 3323.5]     6
(3323.5, 3653.85]     2
(3653.85, 3984.2]     2
(3984.2, 4314.55]     2
(4314.55, 4644.9]     2
(4644.9, 4975.25]     1
(4975.25, 5305.6]     0
(5305.6, 5635.95]     2
(5635.95, 5966.3]     0
(5966.3, 6296.65]     0
(6296.65, 6627.0]     1
Name: count, dtype: int64
Frequency distribution of the Discrete column:  bedrooms
bedrooms
2    19
3    64
4    15
5     2
Name: count, dtype: int64
Frequency distribution of the Discrete column:  baths
baths
1    17
2    71
3    11
4     1
Name: count, dtype: int64
```



(b) Find the percentage of observations that fall within one standard deviation of the mean. Why is this not close to 68%?

We already figured out that the shape of the price is right skewed. As per the book, the 68%, 95%, All of the samples rule is applicable only for bell-shaped distributions.

In [194…
```
# Assuming that the ask is for the 'price' variable

mean = df_house['price'].mean()
std =  df_house['price'].std()
lower_bound = mean - std
upper_bound = mean + std

filtered_data = df_house[(df_house['price'] >= lower_bound )
                         & (df_house['price'] <= upper_bound)]
print('Percentage of filtered data', len(filtered_data)/len(df_house)*100)
```

```
Percentage of filtered data 85.0
```

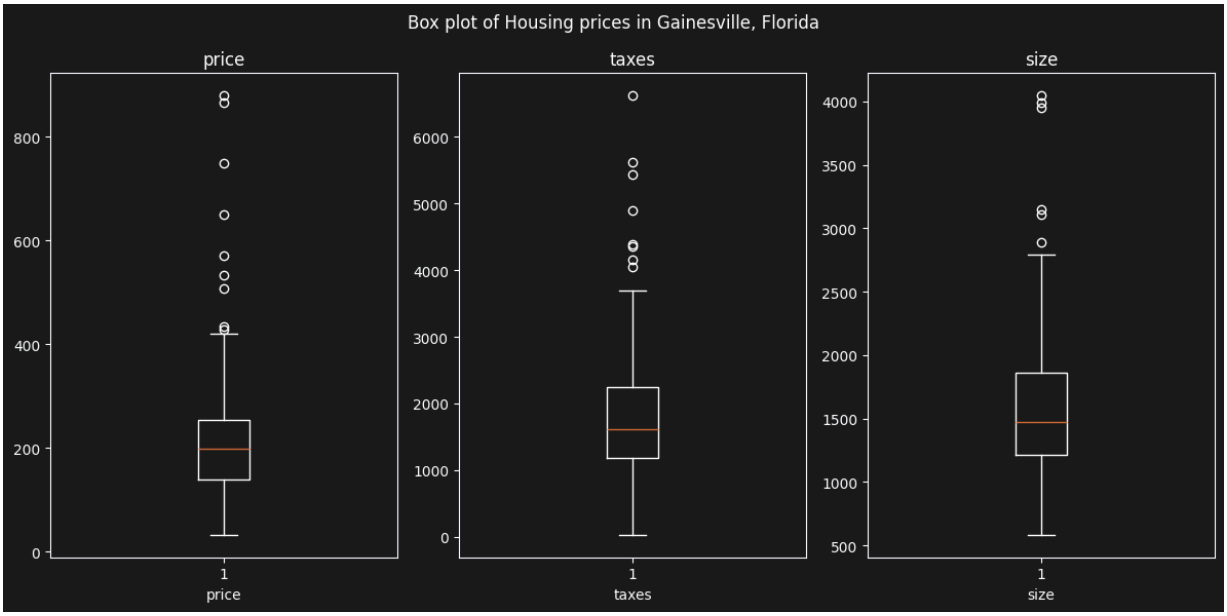
(c) Construct a box plot, and interpret.

The boxplots for price, taxes, and size all show right-skewed distributions with several outliers. In each case, the upper whisker is longer than the lower whisker, and multiple observations lie above the upper whisker, indicating the presence of unusually large houses, taxes, and prices.

The selling price distribution has several expensive outlier homes, suggesting that a small number of houses are priced substantially higher than the majority. Similarly, taxes exhibit strong variability and extreme outliers, likely corresponding to high value properties. House size also shows a right-skewed pattern with a few unusually large homes compared to the typical house size in the dataset.

Looking at the median, it suggests that the shape within the box is symmetric for price but not for taxes and size

In [203…
```
#Assuming again that the ask if for the 'price' variable, but adding it for

continuous_columns =['price', 'taxes','size']

gs = gridspec.GridSpec(1,3)
fig = plt.figure(figsize=(12,6))
fig.suptitle('Box plot of Housing prices in Gainesville, Florida')

for i in range(0,3):
    ax = fig.add_subplot(gs[i])
    ax.set_xlabel(continuous_columns[i])
    ax.set_title(continuous_columns[i])
    ax.boxplot(df_house[continuous_columns[i]])

fig.tight_layout()
```



(d) Use descriptive statistics to compare selling prices according to whether the house is new.

The average selling price of newer homes is significantly higher than that of older homes, with the mean price of new houses being approximately twice as high. The price variation within their group is also twice for new house compared to old ones (based on std) About 50% of old houses are priced between 135k and 240k, whereas 50% of new houses are priced between approximately 257k and 520k (based on the IQR).

Inteestingly, the most expensive house in the given dataset seems to be an older house

In [207…

```
## DISCLAIMER : I needed to look up how to group things 2 variables in Panda

current_set = df_house.groupby('new')['price']

# print (current_set)
# current_set.head() - interesting, I was expecting 2 rows with cumulative c
current_set.describe().T
# 1 is new house
```

Out[207...

new	0	1
count	89.000000	11.000000
mean	207.851124	436.445455
std	121.039149	219.832789
min	31.500000	158.850000
25%	135.000000	256.950000
50%	190.800000	427.500000
75%	240.000000	519.675000
max	880.500000	866.250000

AI Problem: Preparing Data for Healthcare Predictions

AI in healthcare thrives on clean, insightful data prep: models like random forests predict gallstones from features like BMI and age, but garbage in = garbage out. Here, you'll perform EDA on the Gallstone-1 dataset to spot patterns, directly informing AI steps like feature scaling or class balancing. This is your first taste of how stats prevent biased predictions in tools saving lives, like FDA-approved AI diagnostics flagging risks pre-surgery.

Dataset

UCI's Gallstone-1 [https://www.kaggle.com/datasets/xixama/gallstone-dataset-uci?resource=download]. Load gallstone.xlsx (~320 rows).

Tasks

(a) Feature Exploration Load data. Show first 5 rows and info. Summarize key features and their distributions. Explain what these findings reveal about the data's central tendencies and variability, and how they might inform initial AI preprocessing decisions, such as normalization.

(b) Exploratory Data Analysis Conduct a deeper analysis on key features and their relationship to `gallstone status`. Prepare a brief report on your findings. Include at least 5 clean, well-formatted visualizations.

(c) Data Augmentation Generate 50 synthetic "augmented" rows for gallstone-present cases. Append, re-run your analysis in part (b). How does augmentation smooth variances, mimicking real GANs for underrepresented classes?

Answer:

(a) Feature Exploration Load data. Show first 5 rows and info. Summarize key features and their distributions. Explain what these findings reveal about the data's central tendencies and variability, and how they might inform initial AI preprocessing decisions, such as normalization.

The data may require further validation because some obesity percentage values exceed 100%, which is not medically realistic. The histogram of Obesity (%) compresses most observations into a single region while a few extreme outliers stretch the scale significantly. These unusually large values could negatively affect AI model training and preprocessing steps.

The sample has approximately equal distribution of gender, and response variable state.

Several clinical measurements such as glucose, triglycerides, AST, ALT, and CRP appear strongly right-skewed with outliers. Such distributions may disproportionately influence machine learning models.

Many physiological variables such as age, height, BMI, hemoglobin, and cholesterol appear closer to bell shaped distributions with moderate variability

Note: Based on the summary statistics, several variables have maximum values that are far larger than their upper quartiles and medians, indicating strong right-skewed distributions and potential outliers. Examples include glucose, triglycerides, AST, ALT, HDL, and obesity percentage. For many variables, the mean is noticeably higher than the median, which further suggests right-skewness caused by extreme high values.

There may be additional anomalous values in other variables similar to the obesity %. The anomaly in Obesity (%) was easier to identify because percentage values beyond typical ranges are visually noticeable in the histogram. A subject matter expert with deeper knowledge of the expected ranges for these clinical measurements may be able to identify additional unusual observations or potential data quality issues from the distributions.

DISCLAIMER : looks like I need to setup Kaggle API keys in order to access this data on demand (I would not prefer working with a local file as I beleive it would be more useful to connect to Kaggle programatically for future needs). Used Claude to figure out the steps involved.

I had to look up what EDA actually meant - Exploratory Data Analysis - understanding the data before building models

In [233…

```
# df_gall = pd.read_csv('https://www.kaggle.com/datasets/xixama/gallstone-da

### DISCLAIMER : looks like I need to setup Kaggle API keys in order to acce

## I had to look up what EDA actually meant - Exploratory Data Analysis - un
```

```python
## https://www.kaggle.com/settings/api -> Generate an API key first and stor

## install Kaggale in local machine (based on Claude's instructions)

import kagglehub
import os

# Download latest version
path = kagglehub.dataset_download("xixama/gallstone-dataset-uci")
csv_file = os.path.join(path, "gallstone_.csv")
## adding sep=//s like the other data files we worked so far does not work h
df_gall = pd.read_csv(csv_file)

# show first 5 rows of data
display(df_gall.head())

df_gall.info()## 37 explanatory variables + 1 response variable (binary)

# Descriptive statistics
display(df_gall.describe().T.round(2)) # I prefer nice readable format rathe

# Histograms
df_gall_all_columns = df_gall.columns
df_gall_all_excluded_columns = [
    'Gallstone Status',
    'Gender',
    'Comorbidity',
    'Coronary Artery Disease (CAD)',
    'Hypothyroidism',
    'Hyperlipidemia',
    'Diabetes Mellitus (DM)',
    'Hepatic Fat Accumulation (HFA)'
]

df_gall_all_continuous_columns = [
    col for col in df_gall_all_columns
    if col not in df_gall_all_excluded_columns
]

n_cols = 4
n_rows = (len(df_gall_all_continuous_columns) + n_cols - 1) // n_cols
gs = gridspec.GridSpec(n_rows, n_cols)
fig = plt.figure(figsize=(18, 4 * n_rows))
fig.suptitle('Gallstone Dataset Feature Distributions')

k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(df_gall_all_continuous_columns): ### if total cols is nc
            break
        col = df_gall_all_continuous_columns[k]
        ax = fig.add_subplot(gs[i, j])
        ax.hist(df_gall[col], bins=15)
        ax.set_title(col, fontsize=10)
```

```
        ax.set_xlabel(col)
        ax.set_ylabel('Count')
        ax.tick_params(axis='x', labelsize=8, rotation=45)
        ax.tick_params(axis='y', labelsize=8)
        k += 1
fig.tight_layout()
plt.show()


# Bar graphs for non continuous variables
df_gall_all_categorical_columns = df_gall_all_excluded_columns

n_cols = 4
n_rows = (len(df_gall_all_categorical_columns) + n_cols - 1) // n_cols
gs = gridspec.GridSpec(n_rows, n_cols)
fig = plt.figure(figsize=(18, 4 * n_rows))
fig.suptitle('Gallstone Dataset Non Continuous Feature Distributions')

k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(df_gall_all_categorical_columns): ### if total cols is r
            break
        col = df_gall_all_categorical_columns[k]
        ax = fig.add_subplot(gs[i, j])
        df_gall[col].value_counts().sort_index().plot(kind='bar', ax=ax)
        ax.set_title(col, fontsize=10)
        ax.set_xlabel(col)
        ax.set_ylabel('Count')
        ax.tick_params(axis='x', labelsize=8, rotation=45)
        ax.tick_params(axis='y', labelsize=8)
        k += 1
fig.tight_layout()
plt.show()
```

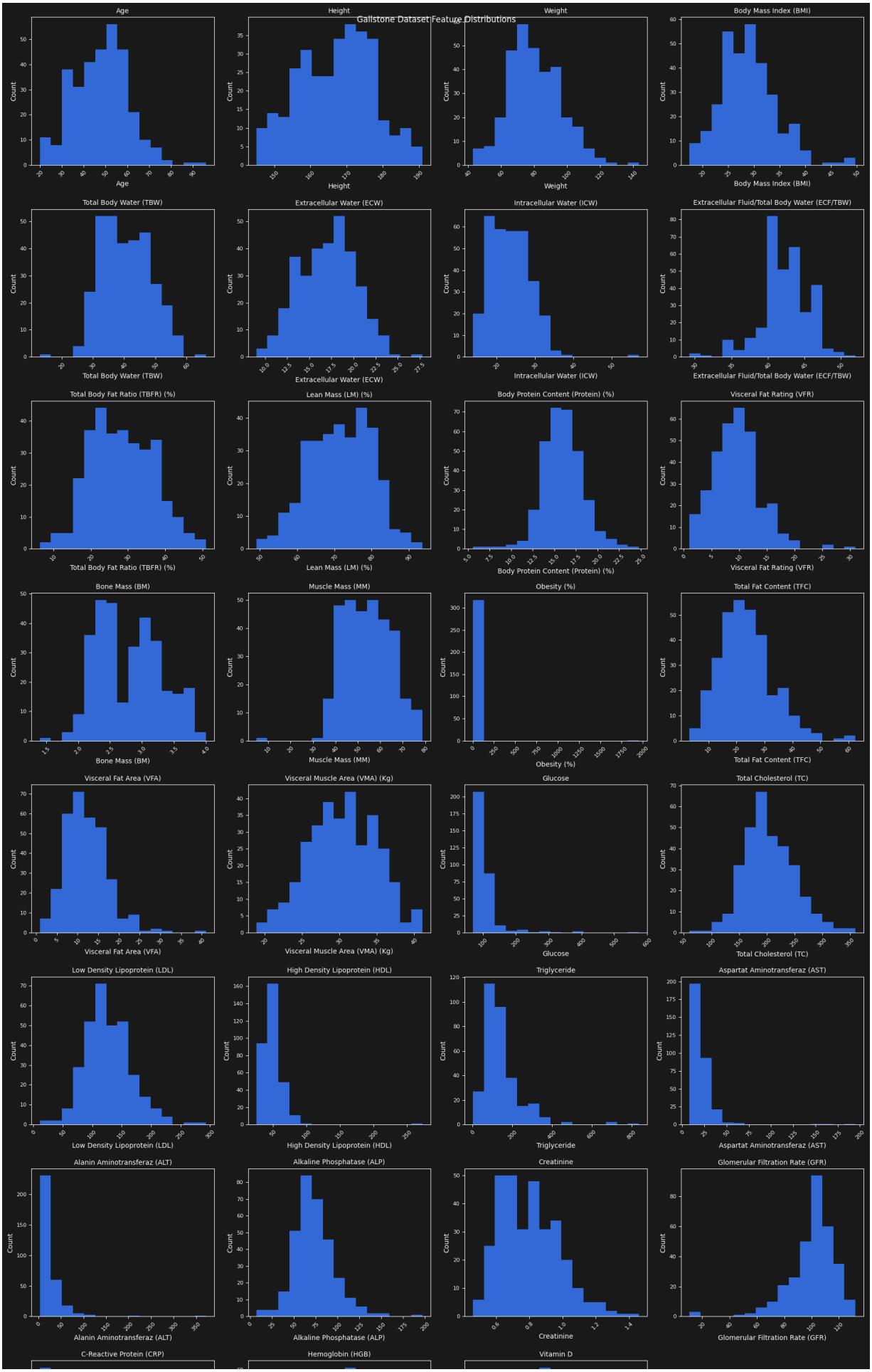
	Gallstone Status	Age	Gender	Comorbidity	Coronary Artery Disease (CAD)	Hypothyroidism	Hyperlipidemia	Di M
0	0	50	0	0	0	0	0	
1	0	47	0	1	0	0	0	
2	0	61	0	0	0	0	0	
3	0	41	0	0	0	0	0	
4	0	42	0	0	0	0	0	

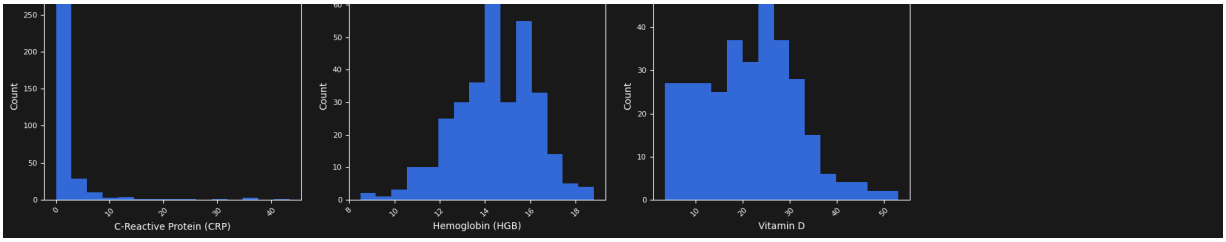
5 rows × 39 columns

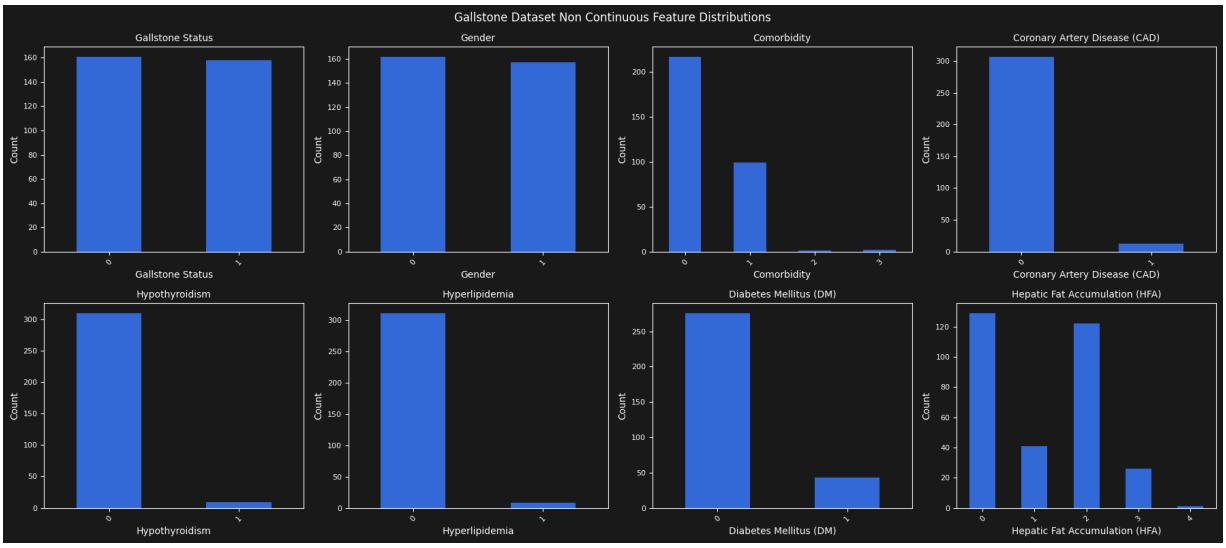
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 319 entries, 0 to 318
Data columns (total 39 columns):
 #   Column                                          Non-Null Count  Dtype  
---  ------                                          --------------  -----  
 0   Gallstone Status                                319 non-null    int64  
 1   Age                                             319 non-null    int64  
 2   Gender                                          319 non-null    int64  
 3   Comorbidity                                     319 non-null    int64  
 4   Coronary Artery Disease (CAD)                   319 non-null    int64  
 5   Hypothyroidism                                  319 non-null    int64  
 6   Hyperlipidemia                                  319 non-null    int64  
 7   Diabetes Mellitus (DM)                          319 non-null    int64  
 8   Height                                          319 non-null    int64  
 9   Weight                                          319 non-null    float64
 10  Body Mass Index (BMI)                           319 non-null    float64
 11  Total Body Water (TBW)                          319 non-null    float64
 12  Extracellular Water (ECW)                       319 non-null    float64
 13  Intracellular Water (ICW)                       319 non-null    float64
 14  Extracellular Fluid/Total Body Water (ECF/TBW)  319 non-null    float64
 15  Total Body Fat Ratio (TBFR) (%)                 319 non-null    float64
 16  Lean Mass (LM) (%)                              319 non-null    float64
 17  Body Protein Content (Protein) (%)              319 non-null    float64
 18  Visceral Fat Rating (VFR)                       319 non-null    int64  
 19  Bone Mass (BM)                                  319 non-null    float64
 20  Muscle Mass (MM)                                319 non-null    float64
 21  Obesity (%)                                     319 non-null    float64
 22  Total Fat Content (TFC)                         319 non-null    float64
 23  Visceral Fat Area (VFA)                         319 non-null    float64
 24  Visceral Muscle Area (VMA) (Kg)                 319 non-null    float64
 25  Hepatic Fat Accumulation (HFA)                  319 non-null    int64  
 26  Glucose                                         319 non-null    float64
 27  Total Cholesterol (TC)                          319 non-null    float64
 28  Low Density Lipoprotein (LDL)                   319 non-null    float64
 29  High Density Lipoprotein (HDL)                  319 non-null    float64
 30  Triglyceride                                    319 non-null    float64
 31  Aspartat Aminotransferaz (AST)                  319 non-null    float64
 32  Alanin Aminotransferaz (ALT)                    319 non-null    float64
 33  Alkaline Phosphatase (ALP)                      319 non-null    float64
 34  Creatinine                                      319 non-null    float64
 35  Glomerular Filtration Rate (GFR)                319 non-null    float64
 36  C-Reactive Protein (CRP)                        319 non-null    float64
 37  Hemoglobin (HGB)                                319 non-null    float64
 38  Vitamin D                                       319 non-null    float64
dtypes: float64(28), int64(11)
memory usage: 97.3 KB
```

	count	mean	std	min	25%	50%	75%	max
Gallstone Status	319.0	0.50	0.50	0.00	0.00	0.00	1.00	1.00
Age	319.0	48.07	12.11	20.00	38.50	49.00	56.00	96.00
Gender	319.0	0.49	0.50	0.00	0.00	0.00	1.00	1.00
Comorbidity	319.0	0.34	0.52	0.00	0.00	0.00	1.00	3.00
Coronary Artery Disease (CAD)	319.0	0.04	0.19	0.00	0.00	0.00	0.00	1.00
Hypothyroidism	319.0	0.03	0.17	0.00	0.00	0.00	0.00	1.00
Hyperlipidemia	319.0	0.03	0.16	0.00	0.00	0.00	0.00	1.00
Diabetes Mellitus (DM)	319.0	0.13	0.34	0.00	0.00	0.00	0.00	1.00
Height	319.0	167.16	10.05	145.00	159.50	168.00	175.00	191.00
Weight	319.0	80.56	15.71	42.90	69.60	78.80	91.25	143.50
Body Mass Index (BMI)	319.0	28.88	5.31	17.40	25.25	28.30	31.85	49.70
Total Body Water (TBW)	319.0	40.59	7.93	13.00	34.20	39.80	47.00	66.20
Extracellular Water (ECW)	319.0	17.07	3.16	9.00	14.80	17.10	19.40	27.80
Intracellular Water (ICW)	319.0	23.63	5.35	13.80	19.30	23.00	27.55	57.10
Extracellular Fluid/Total Body Water (ECF/TBW)	319.0	42.21	3.24	29.23	40.08	42.00	44.00	52.00
Total Body Fat Ratio (TBFR) (%)	319.0	28.27	8.44	6.30	22.02	27.82	34.81	50.92
Lean Mass (LM) (%)	319.0	71.64	8.44	48.99	65.17	72.11	77.85	93.67
Body Protein Content (Protein) (%)	319.0	15.94	2.33	5.56	14.46	15.87	17.43	24.81
Visceral Fat Rating (VFR)	319.0	9.08	4.33	1.00	6.00	9.00	12.00	31.00
Bone Mass (BM)	319.0	2.80	0.51	1.40	2.40	2.80	3.20	4.00
Muscle Mass (MM)	319.0	54.27	10.60	4.70	45.80	53.90	62.60	78.80
Obesity (%)	319.0	35.85	109.80	0.40	13.90	25.60	41.75	1954.00
Total Fat Content (TFC)	319.0	23.49	9.61	3.10	17.00	22.60	28.55	62.50
Visceral Fat Area (VFA)	319.0	12.17	5.26	0.90	8.57	11.59	15.10	41.00

	count	mean	std	min	25%	50%	75%	max
Visceral Muscle Area (VMA) (Kg)	319.0	30.40	4.46	18.90	27.25	30.41	33.80	41.10
Hepatic Fat Accumulation (HFA)	319.0	1.15	1.06	0.00	0.00	1.00	2.00	4.00
Glucose	319.0	108.69	44.85	69.00	92.00	98.00	109.00	575.00
Total Cholesterol (TC)	319.0	203.50	45.76	60.00	172.00	198.00	233.00	360.00
Low Density Lipoprotein (LDL)	319.0	126.65	38.54	11.00	100.50	122.00	151.00	293.00
High Density Lipoprotein (HDL)	319.0	49.48	17.72	25.00	40.00	46.50	56.00	273.00
Triglyceride	319.0	144.50	97.90	1.39	83.00	119.00	172.00	838.00
Aspartat Aminotransferaz (AST)	319.0	21.68	16.70	8.00	15.00	18.00	23.00	195.00
Alanin Aminotransferaz (ALT)	319.0	26.86	27.88	3.00	14.25	19.00	30.00	372.00
Alkaline Phosphatase (ALP)	319.0	73.11	24.18	7.00	58.00	71.00	86.00	197.00
Creatinine	319.0	0.80	0.18	0.46	0.65	0.79	0.92	1.46
Glomerular Filtration Rate (GFR)	319.0	100.82	16.97	10.60	94.17	104.00	110.74	132.00
C-Reactive Protein (CRP)	319.0	1.85	4.99	0.00	0.00	0.22	1.62	43.40
Hemoglobin (HGB)	319.0	14.42	1.78	8.50	13.30	14.40	15.70	18.80
Vitamin D	319.0	21.40	9.98	3.50	13.25	22.00	28.06	53.10


Gallstone Dataset Feature Distributions
Age
Height
Weight
Body Mass Index (BMI)
Total Body Water (TBW)
Extracellular Water (ECW)
Intracellular Water (ICW)
Extracellular Fluid/Total Body Water (ECF/TBW)
Total Body Fat Ratio (TBFR) (%)
Lean Mass (LM) (%)
Body Protein Content (Protein) (%)
Visceral Fat Rating (VFR)
Bone Mass (BM)
Muscle Mass (MM)
Obesity (%)
Total Fat Content (TFC)
Visceral Fat Area (VFA)
Visceral Muscle Area (VMA) (Kg)
Glucose
Total Cholesterol (TC)
Low Density Lipoprotein (LDL)
High Density Lipoprotein (HDL)
Triglyceride
Aspartat Aminotransferaz (AST)
Alanin Aminotransferaz (ALT)
Alkaline Phosphatase (ALP)
Creatinine
Glomerular Filtration Rate (GFR)
C-Reactive Protein (CRP)
Hemoglobin (HGB)
Vitamin D
Count





(b) Exploratory Data Analysis Conduct a deeper analysis on key features and their relationship to `gallstone status`. Prepare a brief report on your findings. Include at least 5 clean, well-formatted visualizations.

The selected features for deeper analysis were picked based on the histograms from part (a). I mainly focused on variables that showed strong skewness, larger spread, visible outliers, or unusual distributions.

The grouped boxplots show that some variables such as BMI, HDL, and CRP have slightly higher median values for the gallstone-positive group compared to the non-gallstone group. However, there is still a lot of overlap between the two groups, so no single variable seems to clearly separate gallstone status by itself.

Glucose, triglyceride, and CRP are strongly right skewed with several large outliers. Obesity (%) contains an extremely large outlier which compresses most of the remaining observations into a very small visible region in the plots. These kinds of extreme values can heavily influence averages and AI models.

The comparative histograms also show that the distributions between the two gallstone groups are not identical. BMI and HDL appear slightly shifted toward higher values for the gallstone-positive group, while triglyceride and CRP show wider spread and heavier tails.

The scatterplots show some relationships between variables. BMI and Obesity (%) show a positive relationship, although the extreme obesity outlier distorts the scale

significantly. Triglyceride and HDL appear to show a mild inverse relationship. Glucose and triglyceride show clustering around lower ranges with a few very large outliers.

The categorical comparison plots show mild differences across some categories. Diabetes Mellitus (DM) and Hyperlipidemia appear slightly more common in the gallstone-positive group. Most other categorical variables appear fairly similar between the two groups.

Overall, the dataset contains: - several strongly skewed variables, - multiple extreme outliers, - variables on very different scales, - overlapping distributions between gallstone groups.

Based on these findings, preprocessing steps such as normalization or standardization may help before training AI models. Outlier handling may also be useful since a few extreme values heavily affect several distributions and visualizations.

DISCLAIMER

The first 4 visualizations were closer to the textbook examples. For the fifth visualization, I explored scatter plots after reading more about EDA techniques. While implementing it, I understood better how scatter plots work with pairs of continuous variables and how grouping can still be visualized using color.

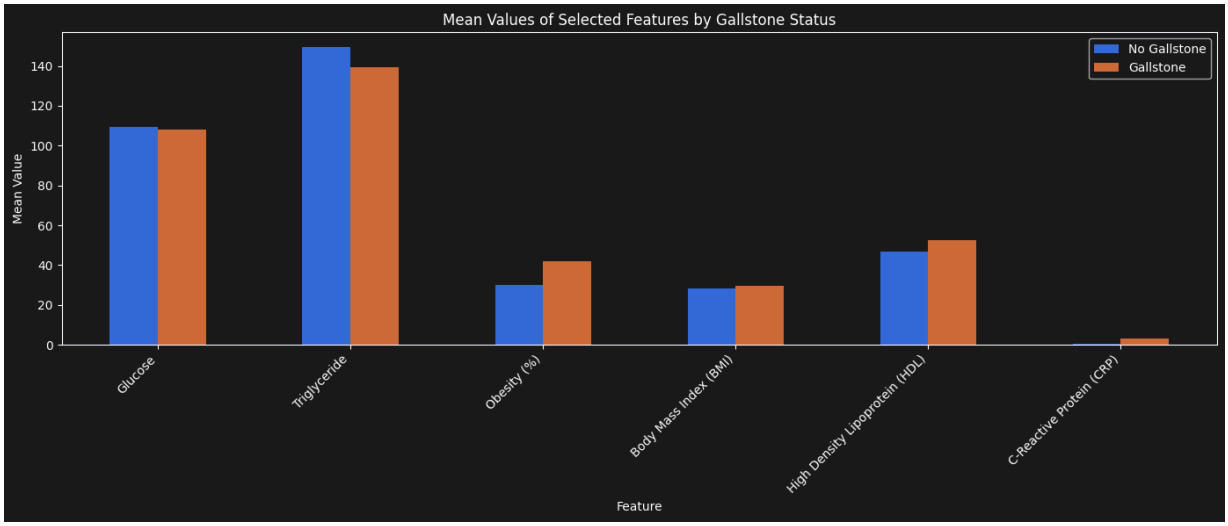
In [244…

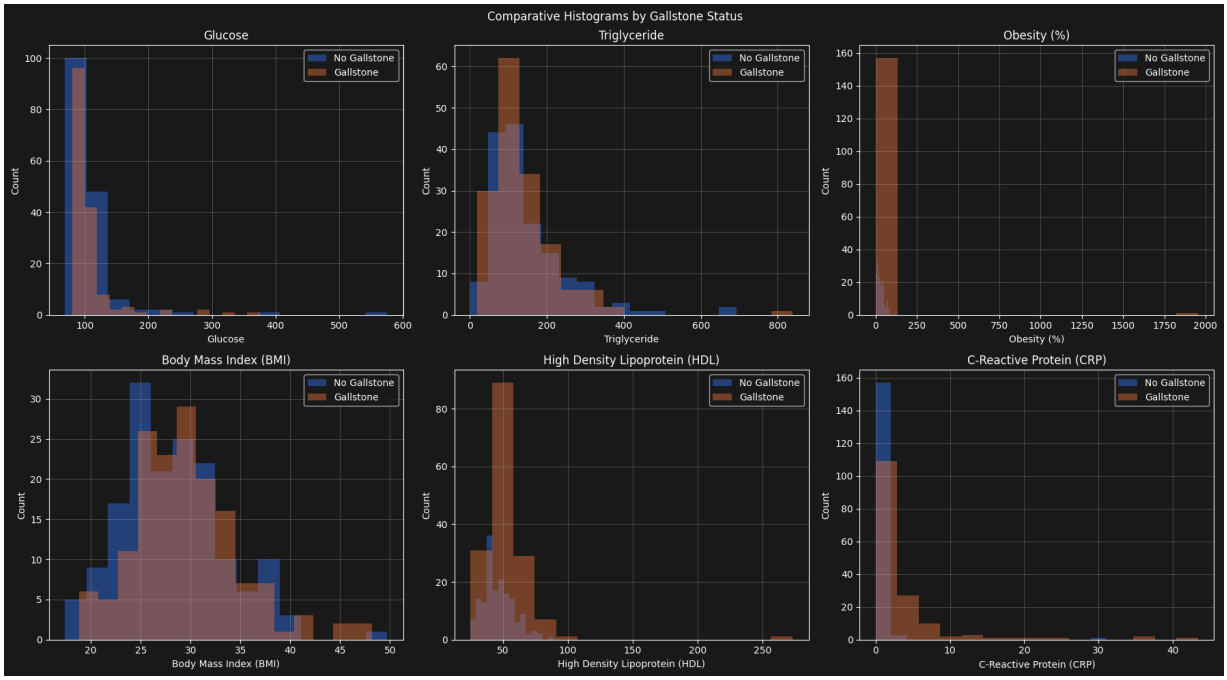
```
# key features ad their relationship with gall stone status
# based on above histograms, picking a few that seems interesting
# Glucose – strong right skew
# Triglyceride – large spread + outliers
# Obesity (%) – anomaly/outlier
# BMI – broader distribution
# HDL – extreme values
# CRP – highly skewed
features = [
    'Glucose',
    'Triglyceride',
    'Obesity (%)',
    'Body Mass Index (BMI)',
    'High Density Lipoprotein (HDL)',
    'C-Reactive Protein (CRP)'
]
# 1. mean comparison of selected features by gallstone status
mean_by_status = df_gall.groupby('Gallstone Status')[features].mean().T
mean_by_status.plot(kind='bar', figsize=(14,6))
plt.title('Mean Values of Selected Features by Gallstone Status')
plt.xlabel('Feature')
plt.ylabel('Mean Value')
plt.xticks(rotation=45, ha='right')
plt.legend(['No Gallstone', 'Gallstone'])
plt.tight_layout()

# 2. comparative histograms of selected features by gallstone status
```

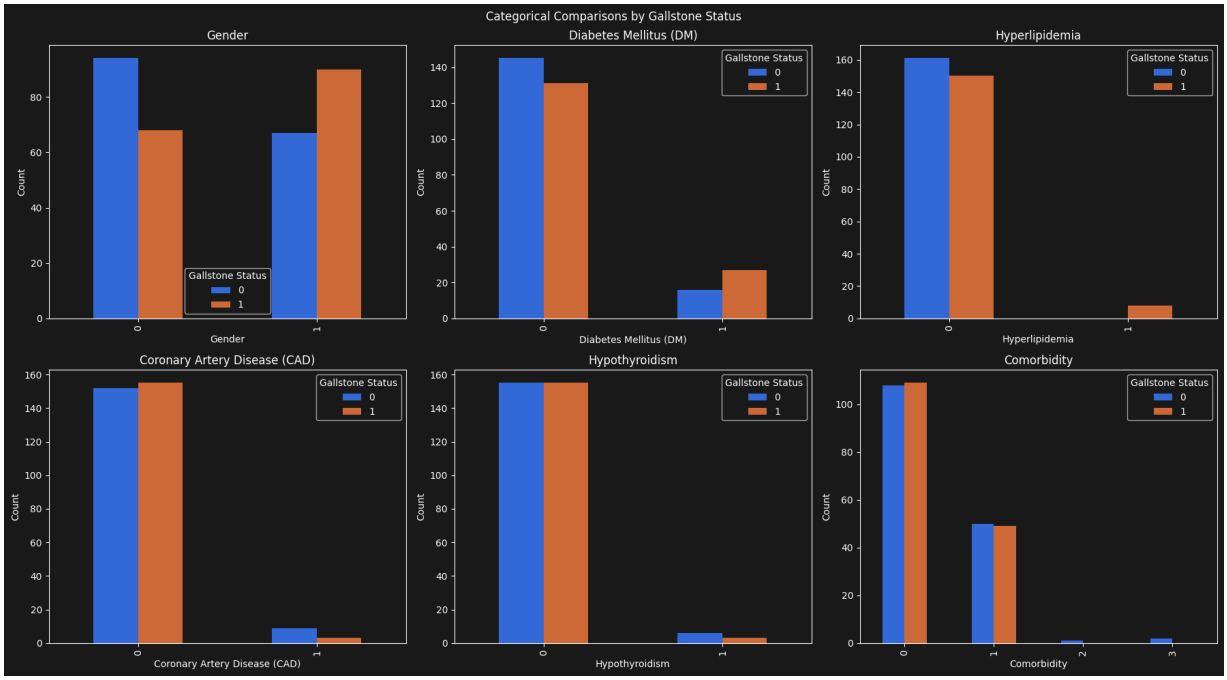
```
n_cols = 3
n_rows = (len(features) + n_cols - 1) // n_cols
fig = plt.figure(figsize=(18,10))
gs = gridspec.GridSpec(n_rows, n_cols)
k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(features): ### if total cols is not a multiple of 3, thi
            break
        ax = fig.add_subplot(gs[i, j])
        feature = features[k]
        df_gall[
            df_gall['Gallstone Status'] == 0
        ][feature].hist(
            bins=15,
            alpha=0.5,
            label='No Gallstone',
            ax=ax
        )
        df_gall[
            df_gall['Gallstone Status'] == 1
        ][feature].hist(
            bins=15,
            alpha=0.5,
            label='Gallstone',
            ax=ax
        )
        ax.set_title(feature)
        ax.set_xlabel(feature)
        ax.set_ylabel('Count')
        ax.legend()
        k += 1
fig.suptitle('Comparative Histograms by Gallstone Status')
plt.tight_layout()


# 3. categorical comparisons by gallstone status
categorical_features = [
    'Gender',
    'Diabetes Mellitus (DM)',
    'Hyperlipidemia',
    'Coronary Artery Disease (CAD)',
    'Hypothyroidism',
    'Comorbidity'
]
n_cols = 3
n_rows = (len(categorical_features) + n_cols - 1) // n_cols
fig = plt.figure(figsize=(18,10))
gs = gridspec.GridSpec(n_rows, n_cols)
k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(categorical_features): ### if total cols is not a multip
            break
        ax = fig.add_subplot(gs[i, j])
        pd.crosstab(
```

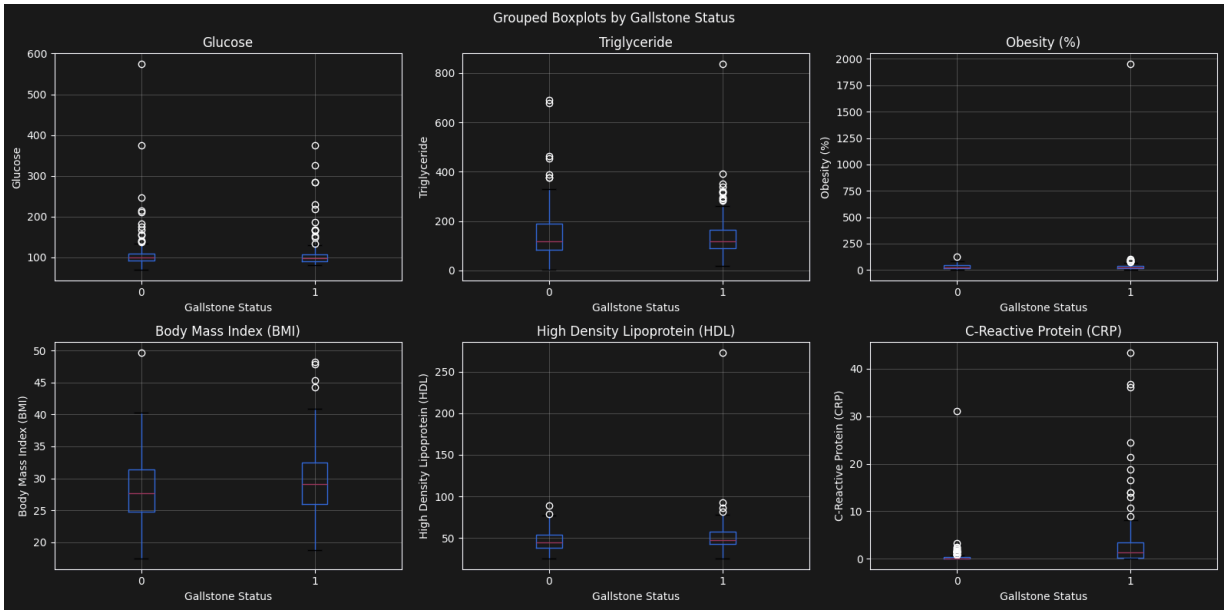
```
            df_gall[categorical_features[k]],
            df_gall['Gallstone Status']
        ).plot( kind='bar',ax=ax)
        ax.set_title(categorical_features[k])
        ax.set_xlabel(categorical_features[k])
        ax.set_ylabel('Count')
        k += 1
fig.suptitle('Categorical Comparisons by Gallstone Status')
plt.tight_layout()


## 4. Box plots
fig = plt.figure(figsize=(16,8))
gs = gridspec.GridSpec(2,3)

k = 0
for i in range(2):
    for j in range(3):
        if k >= len(features):
            break
        ax = fig.add_subplot(gs[i,j])
        df_gall.boxplot(
            column=features[k],
            by='Gallstone Status',
            ax=ax
        )
        ax.set_title(features[k])
        ax.set_xlabel('Gallstone Status')
        ax.set_ylabel(features[k])
        k += 1

fig.suptitle('Grouped Boxplots by Gallstone Status')
plt.tight_layout()
plt.show()


# 5. scatterplots of selected key features
# using the same subset selected from histogram observations
scatter_pairs = [
    ('Glucose', 'Triglyceride'),
    ('Body Mass Index (BMI)', 'Obesity (%)'),
    ('Body Mass Index (BMI)', 'C-Reactive Protein (CRP)'),
    ('Triglyceride', 'High Density Lipoprotein (HDL)')
]

fig = plt.figure(figsize=(16,8))
gs = gridspec.GridSpec(2,2)
k = 0
for i in range(2):
    for j in range(2):
        ax = fig.add_subplot(gs[i,j])
        x_col = scatter_pairs[k][0]
        y_col = scatter_pairs[k][1]
        scatter = ax.scatter(
            df_gall[x_col],
            df_gall[y_col],
```

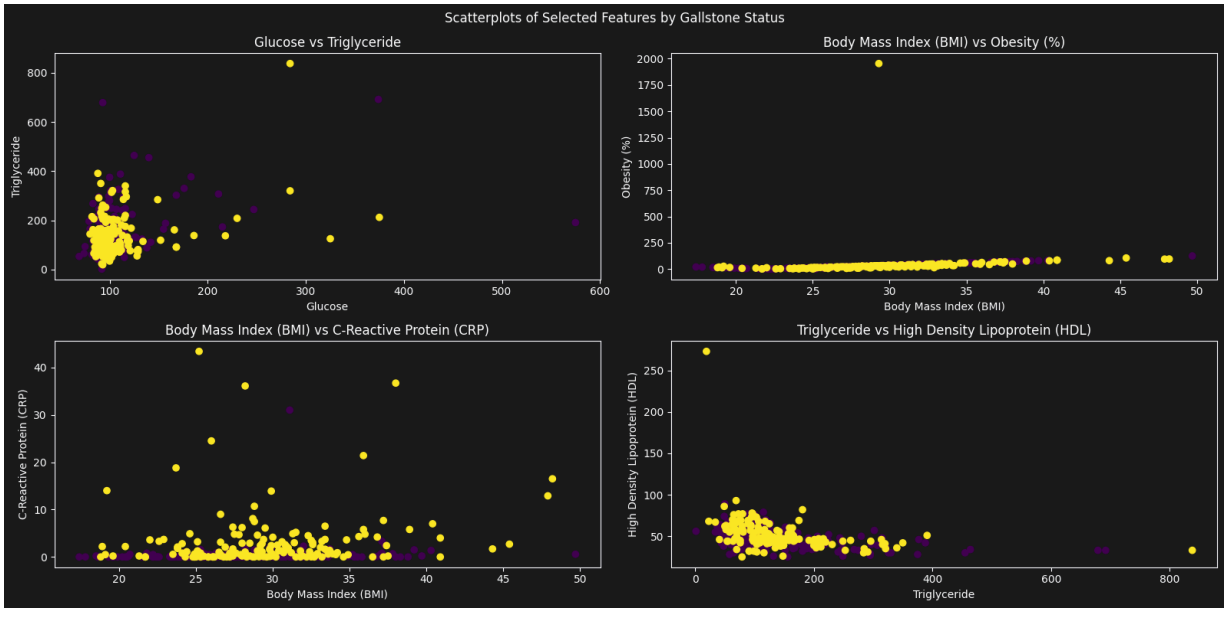
```
            c=df_gall['Gallstone Status']
        )
        ax.set_xlabel(x_col)
        ax.set_ylabel(y_col)
        ax.set_title(f'{x_col} vs {y_col}')
        k += 1
fig.suptitle('Scatterplots of Selected Features by Gallstone Status')
plt.tight_layout()
```






Categorical Comparisons by Gallstone Status
Gender
Diabetes Mellitus (DM)
Hyperlipidemia
Coronary Artery Disease (CAD)
Hypothyroidism
Comorbidity
Count
Gallstone Status
0
1
Gender
Diabetes Mellitus (DM)
Hyperlipidemia
Coronary Artery Disease (CAD)
Hypothyroidism
Comorbidity


Grouped Boxplots by Gallstone Status
Glucose
Triglyceride
Obesity (%)
Body Mass Index (BMI)
High Density Lipoprotein (HDL)
C-Reactive Protein (CRP)
Gallstone Status


Scatterplots of Selected Features by Gallstone Status
Glucose vs Triglyceride
Body Mass Index (BMI) vs Obesity (%)
Body Mass Index (BMI) vs C-Reactive Protein (CRP)
Triglyceride vs High Density Lipoprotein (HDL)
Glucose
Triglyceride
Body Mass Index (BMI)
Obesity (%)
C-Reactive Protein (CRP)
High Density Lipoprotein (HDL)

(c) Data Augmentation Generate 50 synthetic "augmented" rows for gallstone-present cases. Append, re-run your analysis in part (b). How does augmentation smooth variances, mimicking real GANs for underrepresented classes?

DISCLAIMER

I did not know anything about GAN and had to refer multiple online sources including chatGPT and Claude.

Compared to part (b), the biggest visible change is that the Gallstone Status = 1 group now looks denser and smoother in most plots because of the additional synthetic rows.

Some things I noticed:

- The histograms for the gallstone group look a bit smoother now. Before, there were gaps and sudden jumps because this group had fewer samples. After adding more data, the bars look more even.
- The boxplots didn't really change much. The medians and IQRs are almost the same as before. I think this makes sense because the new rows were made from the old ones with just a little bit of noise, so the overall numbers stayed close.
- In the scatter plots, I can see more yellow dots now (these are the gallstone cases). The shape of the clusters looks the same, but the areas where points already existed got more crowded.
- For the categorical plots, the bars for the gallstone group are taller now. This is because I only added new rows to that group, so the class imbalance is not as bad as before.
- The general shape of the data didn't really change. This means the augmentation just filled in more points without making the data look like something totally different.
- The outliers are still there, like that very high obesity value. Since the new samples were created from the existing ones, those weird values still show up and affect some of the plots.

In short, the augmentation did the following:

- Added more samples to the gallstone group
- Made the data less sparse
- Made the distributions look smoother
- Made the plots look a little more stable

This is kind of similar to what a GAN does — it creates fake samples to help the smaller class have more data when training an AI model.

In [248…

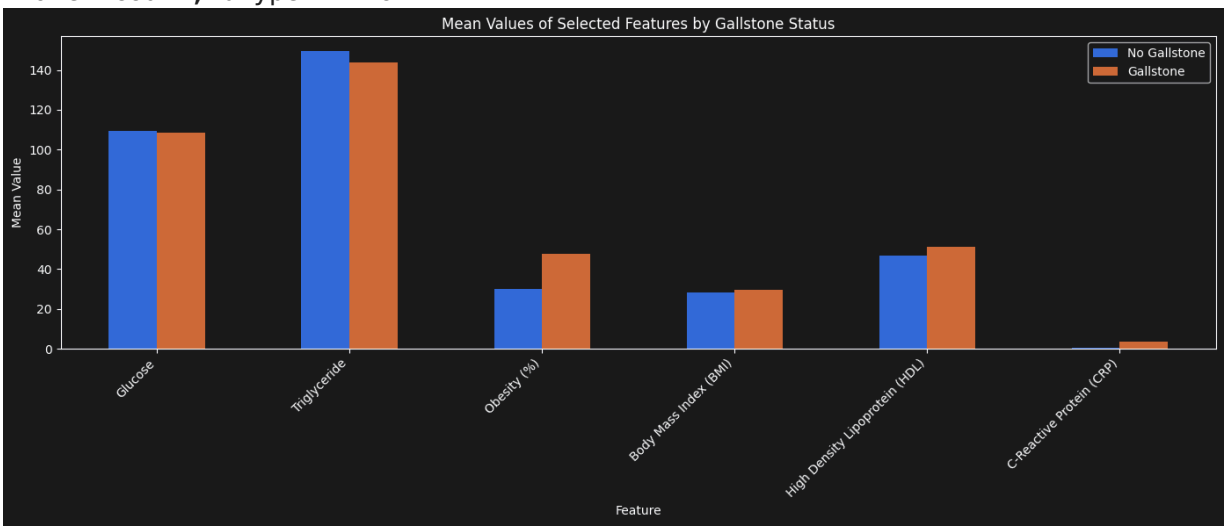
```
# Generate 50 synthetic rows for gallstone-present cases
```

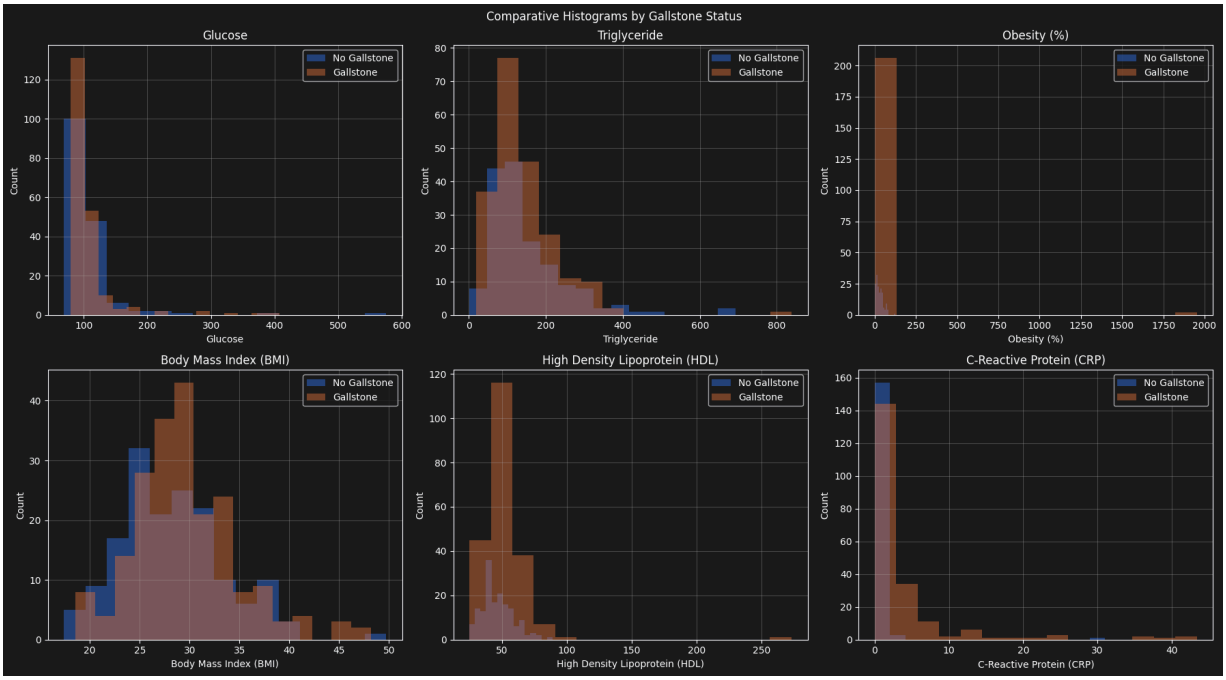
```
gallstone_present = df_gall[df_gall['Gallstone Status'] == 1]
features = [
    'Glucose',
    'Triglyceride',
    'Obesity (%)',
    'Body Mass Index (BMI)',
    'High Density Lipoprotein (HDL)',
    'C-Reactive Protein (CRP)'
]
synthetic_rows = []
# add 50 records in a loop
for i in range(50):
    row = gallstone_present.sample(n=1, replace=True).copy()
    for col in features:
        noise = np.random.normal(  loc=0, scale=row[col].values[0] * 0.05)
        row[col] = row[col] + noise
    row['Gallstone Status'] = 1
    synthetic_rows.append(row)
synthetic_df = pd.concat(synthetic_rows, ignore_index=True)
df_gall_augmented = pd.concat(
    [df_gall, synthetic_df],
    ignore_index=True
)
print("Original shape:", df_gall.shape)
print("Augmented shape:", df_gall_augmented.shape)

print(df_gall['Gallstone Status'].value_counts())
print(df_gall_augmented['Gallstone Status'].value_counts())


# 1. mean comparison of selected features by gallstone status
mean_by_status = df_gall_augmented.groupby('Gallstone Status')[features].mea
mean_by_status.plot(kind='bar', figsize=(14,6))
plt.title('Mean Values of Selected Features by Gallstone Status')
plt.xlabel('Feature')
plt.ylabel('Mean Value')
plt.xticks(rotation=45, ha='right')
plt.legend(['No Gallstone', 'Gallstone'])
plt.tight_layout()

# 2. comparative histograms of selected features by gallstone status
n_cols = 3
n_rows = (len(features) + n_cols - 1) // n_cols
fig = plt.figure(figsize=(18,10))
gs = gridspec.GridSpec(n_rows, n_cols)
k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(features): ### if total cols is not a multiple of 3, thi
            break
        ax = fig.add_subplot(gs[i, j])
        feature = features[k]
        df_gall_augmented[
            df_gall_augmented['Gallstone Status'] == 0
        ][feature].hist(
            bins=15,
```

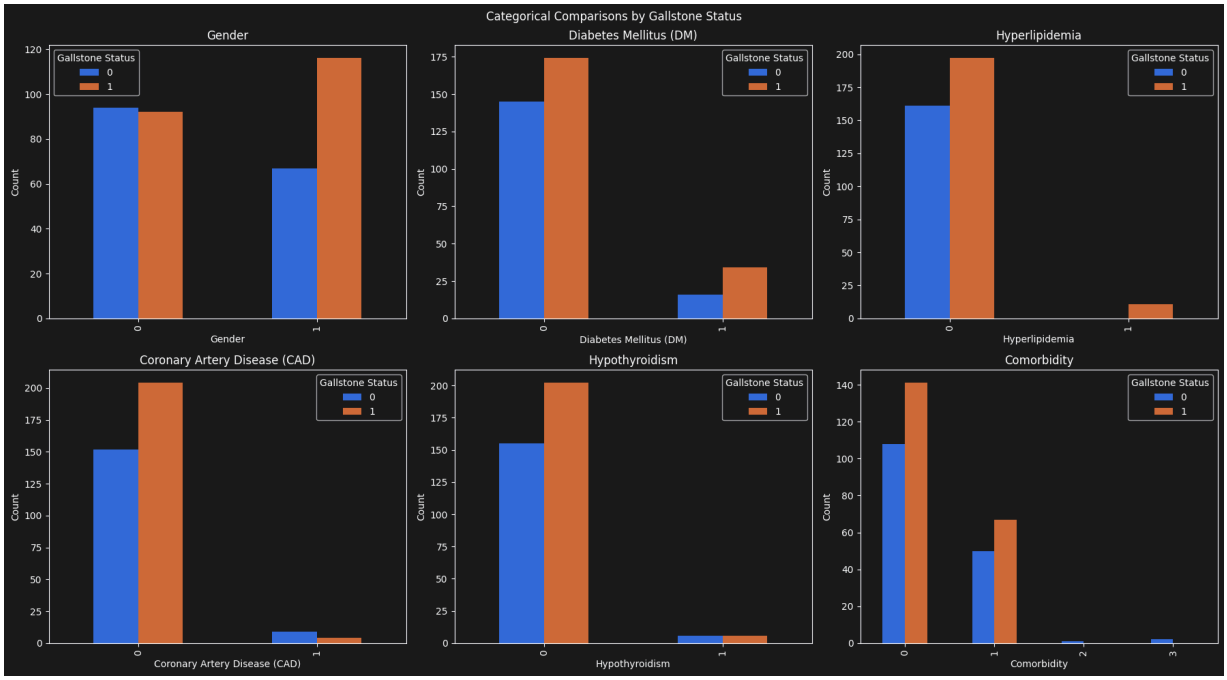
```
            alpha=0.5,
            label='No Gallstone',
            ax=ax
        )
        df_gall_augmented[
            df_gall_augmented['Gallstone Status'] == 1
        ][feature].hist(
            bins=15,
            alpha=0.5,
            label='Gallstone',
            ax=ax
        )
        ax.set_title(feature)
        ax.set_xlabel(feature)
        ax.set_ylabel('Count')
        ax.legend()
        k += 1
fig.suptitle('Comparative Histograms by Gallstone Status')
plt.tight_layout()


# 3. categorical comparisons by gallstone status
categorical_features = [
    'Gender',
    'Diabetes Mellitus (DM)',
    'Hyperlipidemia',
    'Coronary Artery Disease (CAD)',
    'Hypothyroidism',
    'Comorbidity'
]
n_cols = 3
n_rows = (len(categorical_features) + n_cols - 1) // n_cols
fig = plt.figure(figsize=(18,10))
gs = gridspec.GridSpec(n_rows, n_cols)
k = 0
for i in range(n_rows):
    for j in range(n_cols):
        if k >= len(categorical_features): ### if total cols is not a multip
            break
        ax = fig.add_subplot(gs[i, j])
        pd.crosstab(
            df_gall_augmented[categorical_features[k]],
            df_gall_augmented['Gallstone Status']
        ).plot( kind='bar',ax=ax)
        ax.set_title(categorical_features[k])
        ax.set_xlabel(categorical_features[k])
        ax.set_ylabel('Count')
        k += 1
fig.suptitle('Categorical Comparisons by Gallstone Status')
plt.tight_layout()


## 4. Box plots
fig = plt.figure(figsize=(16,8))
gs = gridspec.GridSpec(2,3)
```

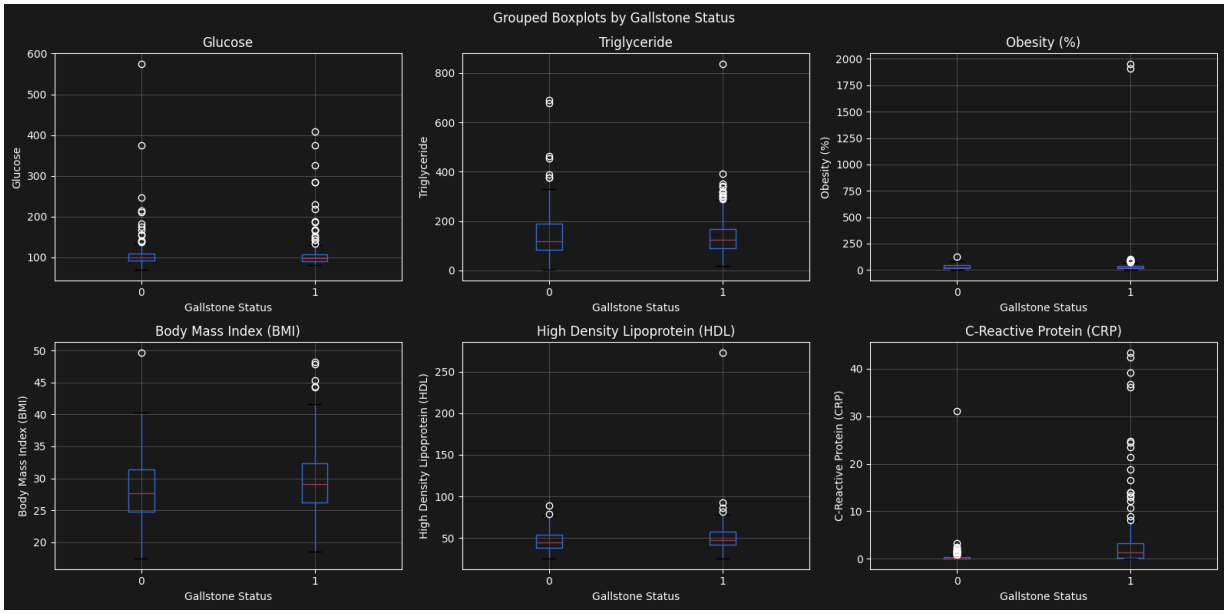
```
k = 0
for i in range(2):
    for j in range(3):
        if k >= len(features):
            break
        ax = fig.add_subplot(gs[i,j])
        df_gall_augmented.boxplot(
            column=features[k],
            by='Gallstone Status',
            ax=ax
        )
        ax.set_title(features[k])
        ax.set_xlabel('Gallstone Status')
        ax.set_ylabel(features[k])
        k += 1

fig.suptitle('Grouped Boxplots by Gallstone Status')
plt.tight_layout()
plt.show()


# 5. scatterplots of selected key features
# using the same subset selected from histogram observations
scatter_pairs = [
    ('Glucose', 'Triglyceride'),
    ('Body Mass Index (BMI)', 'Obesity (%)'),
    ('Body Mass Index (BMI)', 'C-Reactive Protein (CRP)'),
    ('Triglyceride', 'High Density Lipoprotein (HDL)')
]

fig = plt.figure(figsize=(16,8))
gs = gridspec.GridSpec(2,2)
k = 0
for i in range(2):
    for j in range(2):
        ax = fig.add_subplot(gs[i,j])
        x_col = scatter_pairs[k][0]
        y_col = scatter_pairs[k][1]
        scatter = ax.scatter(
            df_gall_augmented[x_col],
            df_gall_augmented[y_col],
            c=df_gall_augmented['Gallstone Status']
        )
        ax.set_xlabel(x_col)
        ax.set_ylabel(y_col)
        ax.set_title(f'{x_col} vs {y_col}')
        k += 1
fig.suptitle('Scatterplots of Selected Features by Gallstone Status')
plt.tight_layout()
plt.show()
```

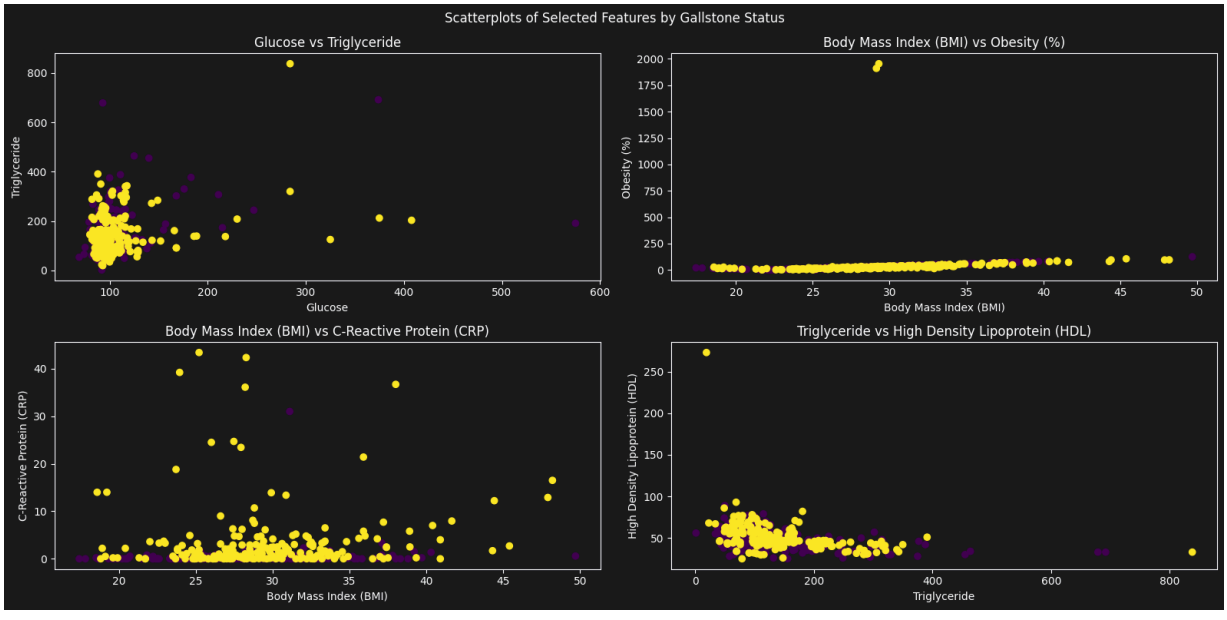
```
Original shape: (319, 39)
Augmented shape: (369, 39)
Gallstone Status
0    161
1    158
Name: count, dtype: int64
Gallstone Status
1    208
0    161
Name: count, dtype: int64
```






Categorical Comparisons by Gallstone Status
Gender
Diabetes Mellitus (DM)
Hyperlipidemia
Coronary Artery Disease (CAD)
Hypothyroidism
Comorbidity
Gallstone Status
0
1
Count


Grouped Boxplots by Gallstone Status
Glucose
Triglyceride
Obesity (%)
Body Mass Index (BMI)
High Density Lipoprotein (HDL)
C-Reactive Protein (CRP)
Gallstone Status


Scatterplots of Selected Features by Gallstone Status
Glucose vs Triglyceride
Body Mass Index (BMI) vs Obesity (%)
Body Mass Index (BMI) vs C-Reactive Protein (CRP)
Triglyceride vs High Density Lipoprotein (HDL)
Glucose
Triglyceride
Body Mass Index (BMI)
Obesity (%)
C-Reactive Protein (CRP)
High Density Lipoprotein (HDL)